

UNIT

4

Introduction to Compiler, Lexical Analysis and Syntax Analysis

SYLLABUS

Introduction: The Structure of a Compiler

Lexical Analysis: The Role of the Lexical Analyzer, Input Buffering, Recognition of Tokens, The Lexical - Analyzer Generator Lex

Syntax Analysis: Introduction, Context-Free Grammars, Writing a Grammar, Top-Down Parsing, Bottom - Up Parsing, Introduction to LR Parsing: Simple LR, More Powerful LR Parsers.

LEARNING OBJECTIVES

- ✓ Overview of the Structure of Compiler
- ✓ The Role of Lexical Analyzer
- ✓ Concept of Input Buffering
- ✓ Process of Recognizing Tokens
- ✓ Designing of Lexical Analyzer Generator
- ✓ Introduction to Syntax Analyzer
- ✓ Basic concept of Context Free Grammar
- ✓ Concept of Top-Down and Bottom-up Parsing
- ✓ Basic Concepts of LR-Parsing Like Simple LR, More Powerful LR Parsing.

INTRODUCTION

Language processing system is a system that translates the source language which is taken as an input into machine. This translation can be done by dividing the source file into module. Compiler is one such module that translates a source program into a target program. This translation is done based on various phases of a compiler. These phases are lexical analyzer (scanner), syntax analyzer (parser), semantic analyzer, Intermediate code generator, code optimizer and code generator. The only phase of a compiler that reads or scans the source program is the scanner or lexical analyzer. The scanning is carried out on a character-by-character basis, thus buffering is a crucial step in making the lexical analyzer to cope up with the speed of other phases. However, LEX program is a tool that is used to generate lexical analyzer or scanner.

The parser or syntax analyzer is the second phase of a compiler. It takes the tokens generated by the scanner and checks the syntax of the input stream line by line according to the given language and then generates a syntax tree. Parsing can be done in two ways i.e., top-down parsing and bottom-up parsing. A top-down parsing may be considered as an attempt to build a parse tree for an input string in preorder, that is starting from the root and then creating nodes of parse tree. Whereas, in bottom-up parsing technology parser tree is constructed by starting from leaf and reaching to the root. Moreover, a bottom-up parsing technique called LR(k) parsing is used to parse a large class of context free grammars.

4.1 INTRODUCTION

Q1. Differentiate between compiler and interpreter.

Answer :

Compiler

A compiler is a program that converts a source program into a target program.

Interpreter

An interpreter is also a program that reads the source program and executes the program line by line.

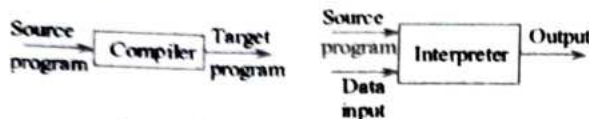


Figure: Compiler and an Interpreter

Difference between Compiler and Interpreter

- ❖ Compiler converts or translates the program from one form of language to another form. An interpreter reads the line, checks the errors if any and reports it to the user. If no errors detected then gives the output.
- ❖ A compiler does not produce output, whereas an interpreter will do so line by line.
- ❖ A program which is compiled, needs to be run by using another program to produce output. Whereas an interpreter takes the source program along data and produces output line by line.
- ❖ A compiler may need to scan the program twice whereas an interpreter will scan the program once.
- ❖ A compiler is user friendly as it does not disturb the user with errors line after line. It gives a complete set of errors in an understandable way to the user. An interpreter will disturb the user after checking each line if there is any error. Even, error reporting is not done in simple English. It can be understood only by a system level programmer.
- ❖ Compilers are not portable as they produce target code which is specific to a machine. Interpreters are portable as they produce code which is machine independent.
- ❖ A compiled program needs more additional programs like assembler, linker/loader. Whereas, an interpreter does not need any one.
- ❖ Compilers can be implemented easily in any programming language however implementing interpreters is not an easy task.
- ❖ For a program to be executed by using an interpreter, the time taken is more, as the output is produced line by line.
- ❖ In a compiler a programmer can use trial and error methods to correct the errors. Whereas in an interpreter the programmer needs to correct the incorrect line and only then the interpreter will check the next line.

4.1.1 The Structure of a Compiler

Q2. With a neat diagram, explain the phases of a compiler in detail.

Answer :

Phases of a Compiler

The phases of a compiler are as follows,

1. Lexical analyzer (scanner)
2. Syntax analyzer (parser)
3. Semantic analyzer
4. Intermediate code generator
5. Code optimizer
6. Code generator.

These six phases perform their specified tasks and two additional activities.

- (i) Symbol table contact or interaction
- (ii) Error handling.

The block diagram comprising all six phases and two additional activities is shown in figure below. The compiler takes input program, performs the necessary actions and generates an output program.

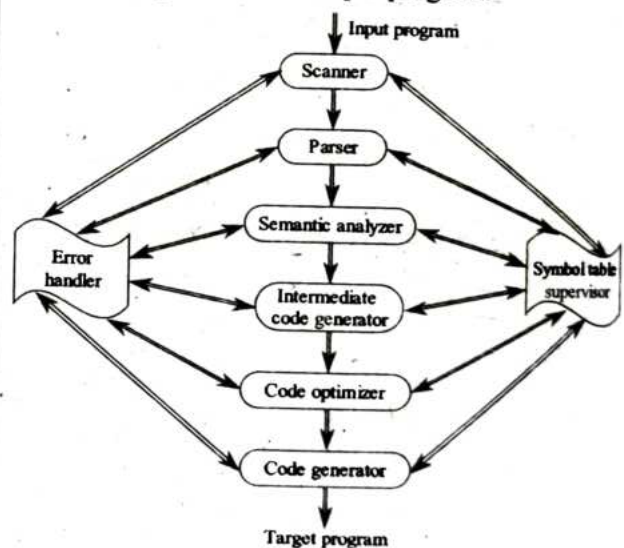


Figure: A General Compiler Format

To describe how a scanner works, the following things must be known,

- (i) **Lexeme:** Lexeme is a smallest logical part of a program.
- (ii) **Token:** A token is a class defined for similar lexemes of a program. For example, keyword, identifier.
- (iii) **Pattern:** To check whether a token is valid in a language, a pattern is used. Hence, pattern is a rule for describing a token. For example, the sentence "an integer can be eight characters long and can have only digits" is a pattern for token "number".

1. Lexical Analyzer (Scanner)

The scanner reads the input string and generates a series of tokens which will be used by the parser. Thus, the scanner can only convert an input string into tokens, however it cannot check whether the tokens give any meaning and are they syntactically correct according to the grammar of a language.

Example

```
while(x < 10)
```

```
{
    .....
    .....
    .....
}
```

In the above piece of code, the scanner cannot identify or report that while is a misspelled form of while in C language. It treats while to be a valid identifier, and generates a token for the same with a pointer to symbol table.

i.e., <identifier, pointer to symbol table for while>

However, the scanner can identify any extra space, new line character and delete them. It also identifies any invalid special character and reports to the user about the same.

Generation of tokens is the main function of a scanner. However, it also interacts with symbol table whenever necessary and report errors when encountered.

Now, when coming to error reporting, the scanner detects, locates, tries to repair and report errors.

2. Syntax Analyzer (Parser)

The parser takes the tokens generated by the scanner and checks the syntax of the input stream line by line according to the given language and then generates a syntax tree. Consider the previous code,

```
while(x < 10)
```

```
{
    .....
    .....
    .....
}
```

The parser will detect an error when it reads the first token in line number 1. Because according to the syntax in C language a procedure cannot have condition as its parameter. Hence, it reports the error stating that "while" is not a valid predefined function.

3. Semantic Analyzer

It also checks whether the input gives any meaning or not. The syntax analyzer looks for semantical errors. In addition to this it also performs type checking, which means it checks that an operation is performed on operands of the same type as specified by the language (input). The semantic analysis phase also performs type coercion (that is automatic type conversion).

Example

```
int a = 12, b = 10;
```

```
float c = 0.0;
```

```
c = a + b; //type coercion is performed here
```

The semantic analyzer will convert the integer variables 'a' and 'b' to real type as the destination is of real type.

4. Intermediate Code Generator

The intermediate code generator performs a conversion of syntax tree into a code called target code. The intermediate code should be easily transferred to target code and intermediate code generation should not take much time.

The intermediate code can be of three forms,

- Postfix
- Triples
- Quadruples.

The most general and commonly used is "three address code". In this type of code, each instruction has atmost three operands.

The intermediate code should have the following basic properties,

- Each instruction can have only one operator in addition to the assignment operator.
- Compiler should generate temporary locations to store the value evaluated in each instruction.
- The code (3 address code) may have less than 3 operands in an instruction.

5. Code Optimizer

The code generated by the 4th phase is optimized to the extent possible by the code optimizer. However, while doing so it does not change the meaning of the program. The level of optimization performed on the code differs from compilers to compilers. The compiler that spends time lavishly on this phase is called as optimizing compilers.

Commonly used code optimizing techniques are,

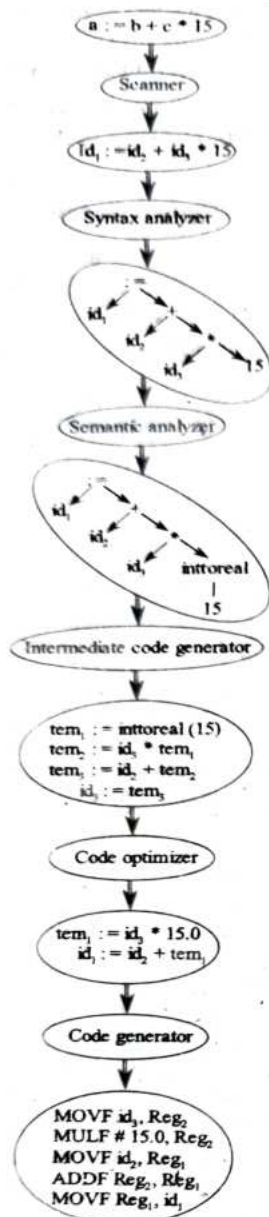
- Common subexpression elimination
- Copy propagation
- Dead code elimination
- Constant folding.

6. Code Generator

Code generation is the last but most important phase of a compiler. It converts the intermediate code into machine or assembly code. Allocation of memory to variables in source program is also done by this phase. The code generator assigns registers to variables in the program.

Example

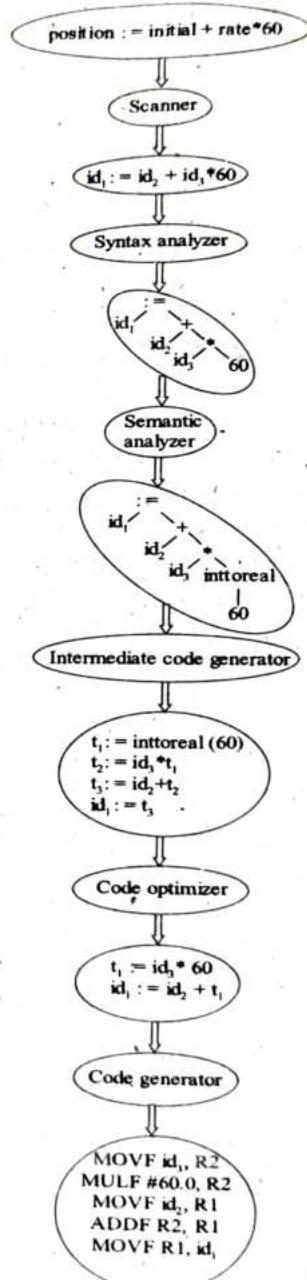
Consider a statement in C language,



The translated statement is shown in figure. The input is scanned by the scanner or lexical analyzer. It recognizes position, initial and rate as valid tokens. Not only this, it also makes an entry into the symbol table for each token, provided that they do not already exist in the symbol table. id_1 , id_2 and id_3 are the internal representations of the tokens position, initial and rate respectively.

The parser or syntax analyzer takes the output from the scanner, checks the syntax of the statement, and generates a parse or syntax tree.

The syntax tree is taken by the semantic analyzer. It does semantic checks on the tree, and makes necessary corrections in the tree. Thus, as an output semantic analyzer also generates a tree. For the given statement, semantic analyzer adds another node "intoreal" to convert an integer constant to a real number.



- (i) intoreal function is used to convert integer to real. In the statement the variables are all real variables however the constant 15 is not. Hence, the semantic analyzer coerces 15 to 15.0.
- (ii) MOVF means that the operation is on float or real variables.
- (iii) # preceding 15.0 means it is a constant.
- (iv) Reg_1 , Reg_2 are the registers.

Q3. Show the translation made by each of the phases of the compiler for the statement $position = initial + rate * 60$. Where position, initial and rate are real numbers.

Answer :

The given statement is, $position := initial + rate * 60$

Figure: Processing Done by the Phases of a Compiler

The intermediate code generator converts the tree from semantic analyzer to an intermediate representation of the source program. For this, it uses temporary variables. For the given statement, the intermediate code generator uses three temporary variables t_1 , t_2 and t_3 .

An optional phase in the compiler is the code optimizer. Its job is to optimize the code. Optimization refers to minimize the number of instructions, operations, variables or temporaries etc., without changing the meaning of the statements generated by the intermediate code generator. In figure, the number of temporary variables is reduced from three to one. Moreover, the 'interior' function is avoided and the value 60 is directly used as 60.0. The number of instructions is also reduced from four to two.

The final phase in the compiler is the code generator. It generates assembly code or relocatable machine code. Since, the given instruction is a floating-point type, the code generator uses float version of the instructions such as MOV, MUL and ADD. A constant is represented with a preceding '#'. The most important task done by the code generator is assigning variables to registers.

Q4. Explain compiler construction tools.

Answer :

A compiler makes use of some specialized tools in order to implement different phases of compilation. These specialized tools are also known as compiler-constructor or compiler-generators or translator writing system. These tools provide automation in compiler design.

The following are some of the compiler-construction tools,

1. Scanner generators
2. Parser generators
3. Syntax-directed translation engines
4. Code-generators
5. Data-flow analysis engines and
6. Compiler-construction toolkits.

1. Scanner Generators

It takes input in the form of regular expression and generates output as a lexical analyzer. For example, LEX scanner generator used in Unix.

2. Parser Generators

It takes input in the form of context free grammar and generates output as a syntax analyzer. For example, YACC parser generator used in Unix.

3. Syntax-directed Translation Engines

It takes input in the form of parse tree and generates output as an intermediate code by translating each node present in the parse tree.

4. Code-generators

It takes input in the form of intermediate code and translates into its equivalent machine code by making use of template matching technique.

5. Data-flow Analysis Engines

It helps in collecting information regarding the flow of values from one part of a program to another part of a program. This is considered as an crucial step in performing good code optimization.

6. Compiler Construction Toolkits

It offers an integrated set of routines in building different compiler's phases.

4.2 LEXICAL ANALYSIS

4.2.1 The Role of the Lexical Analyzer

Q5. What are the functions of lexical analyzer?
OR

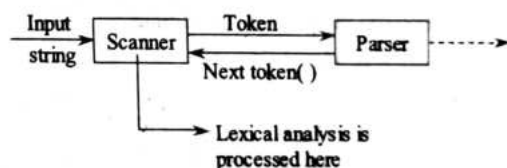
Explain the role of lexical analyzer with its implementations.

Answer :

Lexical Analyzer

Lexical analysis is the process of reading the input string, identifying tokens, deleting white spaces and locating, repairing and reporting errors if there are any.

The lexical analysis is implemented by the first phase of a compiler.



The scanner of a lexical analyzer is responsible for,

1. Removal of white spaces and comments
2. Identifying
 - (i) Constants
 - (ii) Identifiers
 - (iii) Keywords.
3. Generating tokens
4. Reporting about errors.

1. Removal of White Spaces and Comments

Whenever the scanner encounters a blank, tab or a newline character it blindly deletes it. The reason is the parser need not worry about white spaces, as it itself is a complex phase. The parser cannot have regular expression for white spaces. Even comments can be ignored by the parser, as they will be deleted by the scanner, while generating tokens.

2. Identifying:

- (i) **Constants:** Scanner, whenever encounters a constant, it makes an entry into symbol table and return the pointer.

- (ii) **Identifiers:** An identifier can be a name of a function, variable, etc. However, the grammar of any language will consider identifier as a token.

Example: The statement in C,

$a = a + 1;$

The lexical analyzer would consider the above statement as,

iden = iden + number;

Wherein, iden is one token =, +, number are also treated as single tokens.

- (iii) **Keywords:** A keyword in any language will follow all the rules that are imposed on an identifier. The point to be noted is that keywords should be made reserved identifiers so that the scanner do not get confused between an identifier and keyword.

3. Generating Tokens

Whenever, the scanner identifies a valid token, it generates a tuple of the form,

$\langle q_1, q_2 \rangle$

Where, q_1 is a valid lexeme (identifier, number, keyword etc).

q_2 is the pointer to symbol table for q_1 .

The token generation of the lexical analysis phase is initiated by parser by using a function namely next token(). After receiving a signal from parser the scanner starts reading the input string till it finds a valid token.

4. Reporting about Errors

The lexical analyzer in some compilers is responsible for making a copy of source program and marking errors in it. The scanner should be careful about duplication of a single error. However, if the scanner encounters an error, it invokes a procedure in error handler (part of a compiler).

The lexical analysis is completely dependent on the source language and independent of machine, on which the program or source language is being compiled.

Q6. Explain why the analysis portion of a compiler is separated into lexical analysis and parsing phases.

Answer :

The lexical analyzer being the first phase of a compiler reads or scans the input to the compiler and divide the input into a number of small parts of a string (token).

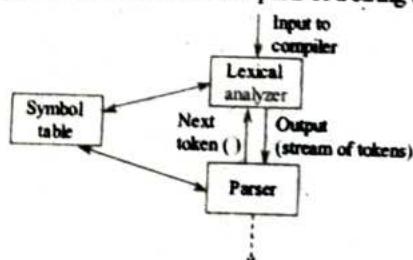


Figure: General Representation of Lexical Analyzer

The above figure clearly shows that the lexical analyzer acts only when it is invoked by the parser.

There are a number of reasons for which the lexical analyzer is separated from syntax analyzer. Some of them are listed below.

Reasons

1. The division of lexical analyzer from syntax analyzer improves the efficiency of the compiler. Each phase is given their responsibilities. "The less the number of tasks, the more is the efficiency" is the motive behind separation. If one phase reads from tokens, check syntax of a string, then the time consumed will be more and task may not be performed efficiently.
2. By the division of lexical analyzer and syntax analyzer into two phases, the designing of a compiler is made very easy. The reason is if these phases are put together then, all the functions that are to be performed by the lexical analyzer has to be performed by syntax analyzer. Out of all these functions performed by lexical analyzer, the most difficult is removing the comments, white spaces, new line characters from the input string. A syntax analyzer is in itself the complicated phase (in terms of designing), which performs syntax checks according to the grammar of any given language. If such a complicated phase performs reading input string, removing spaces looks very absurd. Hence, the division of these two phases makes the design of each phase, in turn design of the compiler easy.
3. Token generation and then storing the input token with its attributes in symbol table is not an easy task. The lexical analyzer not only does token generation, but also makes a copy of the source program and then links with it the number of errors if any.

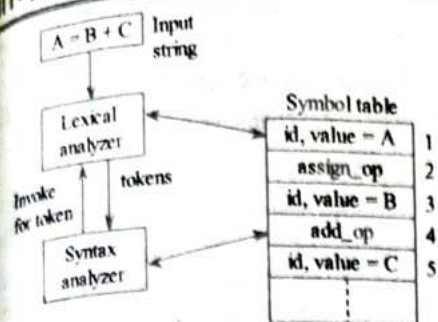
For example, consider the following program in C language,

```
#include <stdio.h>

void main()
{
    printf("Hello!");
}
```

The lexical analyzer reads the input one by one, forms token with attributes and stores it in symbol table. When it encounters '!' it does show an error message as in some compilers the lexical analyzer is responsible for checking so.

Consider an example, on how a lexical analyzer divides a string into tokens, assigns attributes and stores them in symbol table.



The lexical analyzer forms tokens that is any name as in A, B, student is an identifier, operators like add_op, mult_op, div_op are used whenever such operator is encountered.

The format of token given to the parser is,

<id, pointer to symbol table for value >

For identifiers.

<type_op> For operators, 'type' can be any mathematical operator.

Now, generate tokens for the following string,

A = B + C

<id, pointer to symbol table for A>

<assign_op,>

<id, pointer to symbol table for B>

<add_op,>

<id, pointer to symbol table for C>

Hence, the division makes,

- Compiler design easy
- Increases efficiency
- Reduces compilation time.

Moreover, the division also improves portability of the compiler. For example, if any special characters like ↑ are there in Pascal language, the representation is not available, so the character can be isolated. However, if lexical analyzer is used to delete non standard characters then, the compiler will be free from language or device specific restrictions. The parser is not even aware of any language or device specific restrictions, it works normally irrespective of any device specific constraints.

Q7. Define lexeme, token and pattern. Identify the lexeme that make up the token in the following program segment. Indicate the corresponding token and pattern:

```
void swap(int i, int j)
```

```
{ int t;
```

```
t = i;
```

```
i = j;
```

```
j = t;
```

```
}
```

Answer :

Tokens

Token is a sequence of characters that can be treated as a lexical unit in the programming language. It consists of two parts namely **token name** and an attribute value.

Examples of tokens in a programming language include identifiers, keywords, punctuations and operators.

Patterns

Pattern is a rule describing the form of a token. In programming language, pattern is used to determine whether a token is valid or not. Regular expressions are important notations for specifying patterns.

For example, the pattern for token, keyword is sequence of characters that form the keyword and for token, identifier (id) is id → letter "l" (letter "l" digit)*

Lexemes

Lexeme is the smallest logical part of a source program that is matched by the pattern for token. Lexical analyzer recognizes lexeme as an instance of the token.

For instance, token 'RELOP' contains lexemes like ≤, <, >, <=, >=, <>.

Consider the following 'C' statement to better understand the difference between tokens, patterns and lexemes.

```
printf("sum = %d\n", average);
```

In the above statement, **sum** and **average** are lexemes that matches the pattern for token **identifier** and "sum = %d\n" which is enclosed between "" is a lexeme matching the pattern for token, **literal**.

The table below shows some tokens, their corresponding pattern and lexemes.

Token	Pattern	Lexemes
identifier	letter followed by letter or digits	sum, average
number	any number	0, 2.14, 3.26
while	Characters w, h, i, l, e	while
RELOP	<, >, <=, >=, ==, !=	<=, !=
else	Characters e, l, s, e	else

Program

Given program segment is,

```
void swap(int i, int j)
```

```
{
```

```
int t;
```

```
t = i;
```

```
i = j;
```

```
j = t;
```

```
}
```

The table below presents the identified lexemes as well as corresponding tokens and patterns.

Lexemes	Tokens	Patterns
void	keyword	characters v, o, i, d
swap	identifier	letter followed by letter or digits
(	operator	'('
int	keyword	characters i, n, t
i	identifier	letter followed by letter or digits.
,	operator	','
int	keyword	characters i, n, t
j	identifier	letter followed by letter or digits
)	operator	')
{	operator	'{'
int	keyword	characters i, n, t
t	identifier	letter followed by letter or digits
:	operator	':'
t	identifier	letter followed by letter or digits
=	operator	'='
i	identifier	letter followed by letter or digits
:	operator	':'
i	identifier	letter followed by letter or digits
=	operator	'='
j	identifier	letter followed by letter or digits
:	operator	':'
j	identifier	letter followed by letter of digits
=	operator	'='
t	identifier	letter followed by letter or digits
:	operator	':'
}	operator	'}'

Q8. Describe about lexical errors and various error recovery strategies with various examples.

OR

Explain in detail about the role of lexical analyzer with the possible error recovery actions.

Answer :

Errors can be encountered at each phase of computation. Some of them are listed below,

Lexical Errors

A lexical analyzer reads input string and generates tokens. In doing so, it may encounter the following types of errors,

1. A misspelled identifier
2. An identifier with more than specified or defined length.

These errors can be caused due to the following reasons,

- (i) An extra character than specified or defined length
- (ii) An invalid character
- (iii) A missing character
- (iv) Swapped characters or misplaced characters.

The majority of errors that can be identified during the lexical analysis phase of a compiler are misspelled variables or identifiers. The lexical analyzer or scanner without any grammar of a language cannot do more than this. However, while making a copy of source program, it not only marks error but also, provides the line number in which the error has occurred.

Example

Let us consider a statement in C language,
printf("Hello");

In the above statement there are number of errors. printf is not a valid function in C language, however it is a valid identifier. Hence, the scanner cannot detect an error. It considers printf to be a valid identifier. But, this error will be detected during syntax analysis phase. As well as after the '(' there is a missing quotation and the statement is ending with a colon and not a semicolon.

Role of Lexical Analyzer

For answer refer Unit-I, Page No. 99, Q5.

Error Handler of Lexical Analyzer

The error handler of lexical analyzer is shown below,

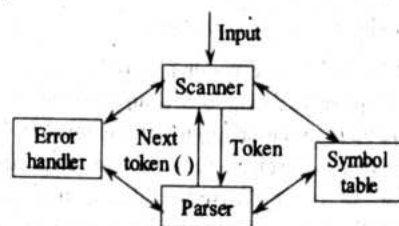


Figure: Error Handler

A compiler not only detects and reports errors but also tries to handle errors to some extent. But this is very expensive process in terms of efficiency and speed, as the compiler cannot repair the error based on what a programmer actually needs. Rather, it can try to repair to the extent possible and give a message to the programmer about the repair as well. For example, if an identifier \$name is declared as an invalid identifier in C language, the lexical analyzer can delete the first character and retain the rest. In the above scenario, the repair was very quick and easy, however in real time scenarios this is not the situation. Errors can be very ambiguous and hard to repair, in such situations the scanner can repair to the possible extent and leave the rest to the programmer.

Repairing an Error

Numeric errors can be handled in an easy manner. The scanner should read a numeric constant and if it exceeds the length then the extra numbers can be truncated and rest retained. The truncation is done from the place where it exceeds the defined length. For example, in C language integer can be of length 2 bytes. If the constant exceeds the size, the scanner will truncate the exceeded portion and inform the same to the programmer.

In case of comments if an extra character appears outside the comment section or boundary, the character will be deleted.

If an illegal or invalid character appears, the same can be deleted.

If the scanner is not able to repair an error and is unable to proceed further, it should not get stuck to the error rather it can delete the characters till it gets a valid token that can be passed to the parser. Such type of error recovery is called parsic mode recovery.

4.2.2 Input Buffering

Q9. What is meant by input buffering? Explain the use of sentinels in recognizing tokens.

Answer :

Input Buffering

The only phase of a compiler that reads or scans the source program is the scanner or lexical analyzer. The scanning is carried out on a character-by-character basis, thus buffering is a crucial step in making the lexical analyzer to cope up with the speed of other phases. The technique used by compilers for reading input and keep it in the buffer is called input buffering.

Input Buffer Scheme

Buffering is used in deciding whether a pattern forms a valid token or not. The lexical analyzer reads the input characters from the buffer. Input buffer scheme is one of the buffering techniques. In this technique, the entire buffer is partitioned into two halves each capable of holding 'C' (number of characters), as shown in the below figure,

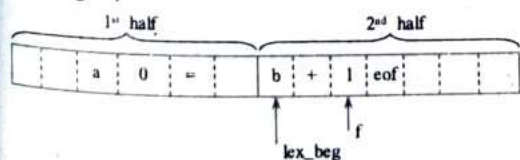


Figure: Two-halves Input Buffer

A single read command is used to read all the 'C' characters from both halves. A special character 'eof' is read into the buffer, if the number of characters in the input are less than C. That is, 'eof' indicates end of source file. This technique uses two pointers *lex_beg* and *f* (forward pointer). The sequence of characters between these pointers give the current lexeme. In the beginning,

lex_beg and *f* point to the same first character. Then, only the pointer *f* is incremented till a lexeme is found. Once a lexeme is found, the pointer *f* is set to point to the next character. When current lexeme is processed, both pointers are set to point to the first character of the next lexeme.

If pointer *f* is reached to the end of the first half of the input buffer, and it is about to point to the first character in the second half of the input buffer, then the second half is immediately filled with 'C' new characters. Similarly, when *f* is about to move out of the second half, the left or first half is filled with 'C' new characters and *f* is made to point to the first character in this half.

The input buffer scheme involves moving the forward pointer '*f*'. An algorithm for advancing the forward pointer '*f*' is as follows,

Algorithm : Advance Forward Pointer

Step1: Check whether '*f*' is at the end of first half. If yes, then goto step (2) else goto step (3).

Step2: Refill the second half and increment '*f*' by 1, i.e., make $f = f + 1$.

Step3: Check whether '*f*' is at the end of second half. If yes, goto step (4), else goto step (5).

Step4: Refill the first half and make '*f*' point to the first character of the first half.

Step5: Increment '*f*' by 1, i.e., make $f = f + 1$.

The two-halve buffering scheme works well for limited lookahead, i.e., it can recognize a token only if characters making a token fit into the same buffer. Programs written in PL/1 are more challenging to these techniques, as the lookahead is too large. For example in the statement,

DECLARE(A1, A2,, Ai)

DECLARE can either mean an array name or a keyword. To decide the exact one, '*f*' has to be moved till the character next to the right parenthesis is reached. Moreover, for every increment of pointer '*f*', two tests need to be performed.

Buffering with Sentinels

The inefficiency of performing two tests for each increment of pointer '*f*' in the input buffer scheme, can be eliminated by using sentinels. A sentinel is a special character usually eof. Sentinels are neither the part of the source program nor counts to the number of characters in the buffer. At the end of each half, a sentinel i.e., eof is used to indicate the end of that half. Moreover, end of the source program is also indicated by another eof. Thus, three eofs are used when the first half is full and the second half is not completely full.

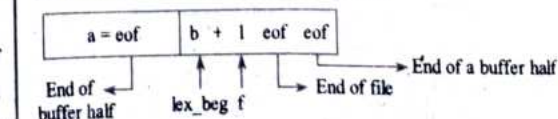


Figure: Input Buffer With Sentinels

With this scheme, most of the times perform only one test to check whether f is pointing to the eof only. When f reaches either of the eofs, i.e., the first half of a buffer, the second half of a buffer or the end of the source program required to perform additional tests. The improved algorithm for incrementing f using sentinels is as follows.

Algorithm : Improved_Advance_Forward_Pointer

Step1: Increment the forward pointer f by 1 i.e., $f = f + 1$.

Step2: Check whether f is pointing to a sentinel i.e., eof. If yes, then goto step (3), else exit.

Step3: Check whether f is at the eof of the first half. If yes, then goto step (4) else goto step (5).

Step4: Refill the second half and increment f by 1.

Step5: Check whether f is pointing to the end of the second half. If yes, then goto step (6), else goto step (7).

Step6: Refill the first half and make f point to the first character in the first half.

Step7: End of the source program is reached, thus, terminate lexical analysis.

4.2.3 Recognition of Tokens

Q10. Write short notes on token specification.

Answer :

Tokens represent a categorized block of text. They can be specified using the following,

1. Strings

For answer refer Unit-I, Page No. 5, Q6(b).

2. Languages

For answer refer Unit-I, Page No. 5, Q6(c).

3. Regular Set

For answer refer Unit-II, Page No. 32, Q3(i).

4. Regular Expression

For answer refer Unit-II, Page No. 32, Q1, Topic: Regular Expression.

Q11. What do you mean by token recognition?

Explain in detail about transition diagram that is used to recognize the token relop (corresponding to relational operators in C++ language).

Answer :

Token Recognition

The process of recognizing tokens in terms of meaningful parts is called token recognition.

Tokens are recognized, iff a pattern is matched with a regular expression. The recognized tokens can be represented in the form of a finite automata and transition diagram.

Initially, the patterns for all the required tokens are considered and a piece of code is created which reads the input string. Then, a prefix is identified that is, a lexeme matching with any one of the pattern.

Transition Diagram

The transition diagram is diagram obtained in the process of transforming a regular expression pattern into a flow chart. This diagram is the collection of nodes and edges, wherein nodes/circles represent states and edges labeled with a symbol or set of symbols show direction from one state to another. Each state in the transition diagram corresponds to a condition in input scanning process, of a lexeme to be found that matches with a regular expression pattern.

For instance, the transition diagram for regular expression, abc^*

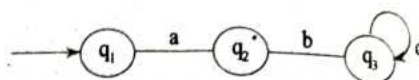


Figure: Transition Diagram

The conventions about transition diagram are as follows,

1. In transition diagram, accepting states or final states are represented by double circle. The final states represent that the required lexeme has been found. Final states include an action if it wants to return an action typically like a token and the corresponding attribute value to the parser.
2. In transition diagram, as there are many transitions there is a possibility of failure to occur. If this happens then forward pointer is retracted back to start state and then the next transition diagram is activated.
3. In transition diagram, an edge marked with start is referred to as "start state" or "initial state". Transition diagram always begins from the "start state" after which only all the subsequent input symbols are read.

Example

For instance, consider the transition diagram identifying lexemes for "relop"

Transition diagram begins at start state, q_0 . If the first input symbol recognized is $>$ then among all the lexemes i.e., $(<, >, <=, >=, <>, =)$ only lexemes $(>=, >)$ are to be recognized.

Transition diagram reads $>$ and enter into state q_1 . If the next character is $=$ then lexeme $>=$ is recognized and the state q_2 is entered where the token and the corresponding attribute value is returned. If in state q_1 the next character is $>$ or $<$ then the pattern identified is either $><$ or $>>$ which are not in relop. So attach a^* to the final state and retract the forward pointer one position (i.e., lexeme does not include the symbol that leads to final state).

If the first character in state q_0 is "=" then this one character must be the lexeme and state q_4 is entered and its corresponding token and attribute is returned.

Finally, if the first character in q_0 is "<" then state q_1 is entered. If in q_1 , the next character is "=" then the lexeme is recognized as "<=" and enter into ' q_6 ' by returning token and attribute value. If "<>" is recognized then state q_5 is entered and token, attribute value is returned. In state q_1 , if other character, lexeme is "<" which do not match with any of relop. So, attach * and retract the forward pointer.

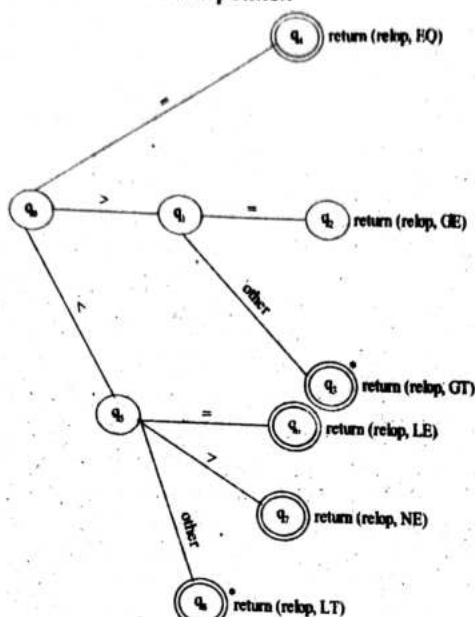


Figure: Transition Diagram for Relop

Q12. What are reserved words and identifiers? Explain how they are recognized with an example.

Answer :

An identifier can be a name of a function, variable, constant etc. A keyword in any language will follow all the rules imposed on an identifier. In Lexical Analysis, keywords are recognized as identifiers. So keywords should be made reserved identifier so that the scanner do not get confused between an identifier and keyword.

For example, If, then and else are keywords, but scanner treats them as identifiers.

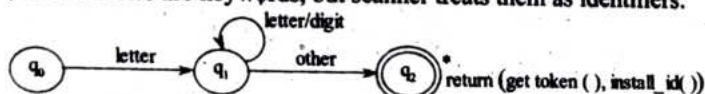


Figure: Transition Diagram for Identifiers and Keywords

The above figure is a transition diagram for searching an identifier lexeme. The diagram also recognizes keywords such as if, then, else as they look as identifiers.

Keywords which look like identifiers can be handled using two methods.

- Initially a symbol table is created in which information about keywords is stored. Two return statements called `install_id()` and `gettoken()` are used in order to obtain attribute value and token respectively. The functionality of `install_id()` is to search for a lexeme in symbol table and make its entry. If the lexeme present in a symbol table is a keyword then `install_id()` returns '0' and if the lexeme found is a variable then `install_id` returns the attribute value i.e., pointer to table entry. If the lexeme is not found in a symbol table then it is placed in symbol table and a pointer to table entry is returned. The function of `gettoken()` is to return the corresponding token i.e., either identifier or keyword for the lexeme found.
- In second technique, a transition diagram is created for each keyword. In this transition diagram, a test for non-letter or digit is applied. For example, consider a transition diagram for keyword "ELSE"

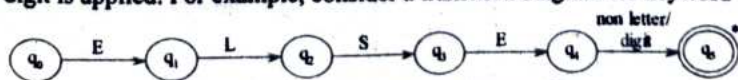


Figure: Transition Diagram for Keyword Else

As seen in the above transition diagram it ends with a state non letter/digit. It means that after any identifier it can be either a digit or a non letter but can not be a character. It is compulsory to check whether identifier has ended or not otherwise a token " ELSE " is returned for the lexeme like elsewhere.

4.2.4 The Lexical - Analyzer Generator Lex

Q13. Write about lexical analyzer generator lex.

Answer :

Lexical Analyzer Generator Lex

LEX program is a tool that is used to generate lexical analyzer or scanner. The tool is often referred as lex compiler and its input specification as lex language.

Lex Specification

Any lex program constitutes three parts or sections in it.

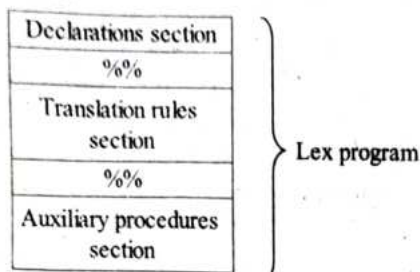


Figure (1): Structure of a Lex Program

Each section is separated by %%.

Declarations Section

This section is used to declare variables, manifest constants and regular definitions.

A manifest constant is an identifier that is used to represent a constant.

Example

In C manifest constants are declared as,

```
#define MAX = 10
```

A regular definition is a sequence of regular expressions where each regular expression is given a name for notational convenience. If $re_1, re_2, \dots, re_m$ is a set of regular expressions used to denote a language and $n_1, n_2, \dots, n_m$ is the set of names for the regular expressions respectively, then a regular definition is given as,

$$\begin{array}{lcl} n_1 & \longrightarrow & re_1 \\ n_2 & \longrightarrow & re_2 \\ & \vdots & \\ n_m & \longrightarrow & re_m \end{array}$$

Translation Rules Section

The translation rules is a set of statements, which defines the action to be performed for each regular expression given. Below is the form of statements in translation rules section.

$$\begin{array}{lcl} RE_1 & & \{method_1\} \\ RE_2 & & \{method_2\} \\ & \vdots & \\ RE_m & & \{method_m\} \end{array}$$

RE_i is the regular expression and $method_i$ is the program, which defines the steps to be taken whenever the lexical analyzer finds that a lexeme matching the regular expression RE_i . These methods are usually implemented in C language, however they can also be implemented in any other language.

Auxiliary Procedures

These procedures are used by the methods in translation rules section. The auxiliary procedures can be compiled separately, however loaded with the scanner.

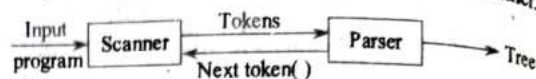


Figure (2): Coordination between Scanner and Parser

Behavior of Lexical Analyzer (If Generated by LEX) with Parser

The process is initiated by parser in the need of tokens. This invocation makes scanner to read the longest input lexeme that exactly match any of the regular expressions RE_i and if the lexeme matches more than one regular expression then the scanner will select the first one. After checking for the match, the respective method or function $method_i$ is called or performed. After executing the method the control should be handed over to the parser, if unable to do so the scanner will scan unless it gets a lexeme whose method (action) will result in a control return to parser. As a result of successful return to parser, the scanner returns a token for a lexeme in the input.

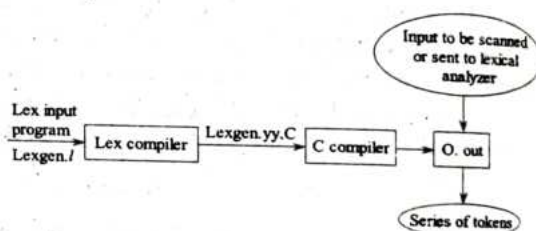


Figure (3): Lex Format to Create a Scanner

The lex compiler takes a Lex program $lexgen.l$ as input and generates $Lexgen.yy.C$, which is a C program. Then this C program ($Lexgen.yy.C$) is compiled through the C-compiler to generate an object code file 'O.out'. To O.out pass the strings or scan the input or to O.out string can be given as input for which tokens need be generated. Hence, O.out becomes the scanner or lexical analyzer.

Q14. Explain with one example how LEX program performs lexical analysis for the following patterns in 'C', identifier, comments, numerical constants, arithmetic operators.

OR

Write a LEX program for identifying the keywords and identifiers from the file?

Answer :

The LEX program for the patterns is given below.

NUM, IDE, AOP

```

%{
word      [letter | digit | -]*
aop       [+ | - | / | * | ^]
delimiter [blank | tab | newline]s
white space [delimiter]+
letter    [a-z A-Z]
digit     [0-9]
identifier {letter}* ({letter} ! {digit})*
num        {digit}* ( \ . {digit}+ )? (E[+-]? {digit}+)*

comments%{ /* (word) | digit | white space)* */
{comments} /*No action is taken, no value
              is returned*/}

{white space} /*No action performed and
              no return value*/}

{identifier} {yLvalue = install_iden( );
              return(IDE);}

{num}        {yLvalue = install_number( );
              return(NUM);}

{aop}        {return(AOP);}

%}

install_iden( )
{
    * This procedure installs the lexeme, which is
    pointed by the pointer text and its length is length,
    in to the symbol table. This procedure returns a
    pointer in to that lexeme*/
}

install_number( )
{
    install_number( ) stores the number in symbol
    table and returns a pointer to its entry.
}

```

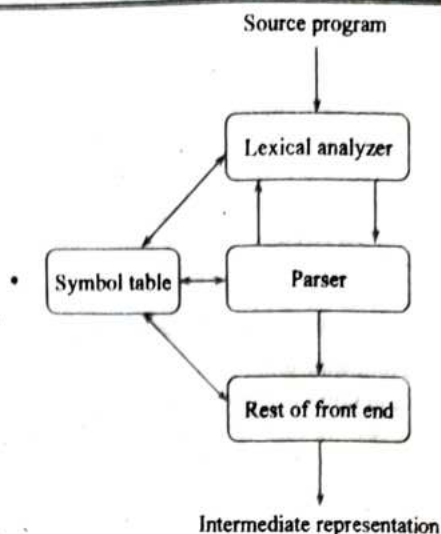


Figure: Role of Parser

(b) Representative Grammars

Some of the grammars that are suitable for specific methods are discussed below,

(i) Expression Grammar

The grammar is called expression grammar, if it describes expressions, terms and factors. For instance, the following grammar is expression grammar.

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E)id$$

Where,

E is the set of expressions. It contains terms which are isolated by + operator.

T is the set of terms. It contains factors which are isolated by * operator.

F is the set of factors. It contains either identifier (id) or parenthesized expression.

This grammar follow associativity and precedence. It is a type of LR grammar suitable for bottom up parsing. Apart from this, it is also useful for dealing with more number of operators and higher levels of precedence. However, it cannot be applied for top-down parsing.

(ii) Constructs

Constructs are grammars that start with keywords such as while, int etc. Such keywords are helpful in selecting the production of the grammar which needs to be considered for matching input. This simplifies the parsing process.

(iii) Non-left Recursive Form of Expression Grammar

The following is a non-left recursive variant of expression grammar

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$\begin{aligned} F &\rightarrow (E)/id, \\ E &\rightarrow TE' \\ E' &\rightarrow +TE' / \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' / \epsilon \\ F &\rightarrow (E)/id \end{aligned}$$

This grammar is suitable for top-down parsing.

(iv) **Handling Ambiguous Grammar**

The following grammar consider plus(+) operator and multiplication(*) operator as identical. Thus, it is used for dealing with ambiguity that arises due to parsing.

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow E * E \\ E &\rightarrow (E) \\ E &\rightarrow id \end{aligned}$$

Where, E represents all types of expressions.

Q16. Explain in detail about syntax error handling.

Answer :

Syntax Error Handling

Following are the concepts associated with syntax error handling.

1. Syntax Error

Syntax or syntactic errors are the errors that arise during syntax analysis. These errors can be incorrect usage of semicolons, additional or missing braces etc. In C or Java, syntactic error could be the presence of a case statement without enclosing switch.

2. Syntax Error Detection

Syntax error can be detected through the use of precise parsing methods. The parsing methods such as LL and LR are capable of detecting the errors immediately after their occurrence. They possess viable-prefix property according to which an error is triggered whenever a prefix of the input string is not considered as a string (according to the language). Alternatively, an error is triggered upon encountering the sequence of tokens from lexical analysis that cannot be parsed any more according to the language grammar.

However, the error handler has to achieve following goals during parsing,

- (i) Address clearly and precisely about the existence of errors. Addressing the error must involve atleast referring the location of an error within the source program.
- (ii) Put low overhead for the processing of error-free programs.
- (iii) Recover from an error as soon as possible so as to detect next errors.

3. Syntax Error Recovery

The various strategies employed by parser to recover from syntactic error are as follows,

- (i) Panic mode recovery
 - (ii) Phrase level recovery
 - (iii) Error production
 - (iv) Global production.
- (i) **Panic Mode Recovery:** Panic mode recovery is a simple strategy employed by most of the parsing method to recover from syntactic error. The idea behind it is to discard a single input symbol one at a time after discovering error until a set of synchronizing tokens like semicolon or end (representing end) are found. Hence this strategy mainly focuses on considerable amount of input skips without checking any additional input error. The advantage of this strategy is that, it ensures that the parser will not enter into infinite loop, while performing the skips. Beside this, method is mainly used when there are less number of errors encountered in the statement.
- (ii) **Phrase Level Recovery:** Phase level recovery strategy is employed by most of error repairing compilers to recover from syntactic error. The main idea in this method is to perform local correction on those input string that were remain unchecked on discovering an error. This correction can be done by substituting a string in place of input prefix. Some of the local corrections done by compiler design are as follows,
- ❖ Replacing comma by semicolon
 - ❖ Deleting extra semicolon
 - ❖ Inserting missing semicolon.
- The main drawback of this method is that it faces difficulty in finding the occurrence of the actual error before performing detection.
- (iii) **Error Production:** Error production strategy helps in generating appropriate error messages that are encountered while parsing a given input string. This method helps in improving the efficiency of parsing by continuously performing parsing on a given input string. One of the main disadvantage of this method is that it faces difficulty in maintaining the grammar. This is because if the grammar changes the corresponding error production need to be changed while parsing.
- (iv) **Global Production:** Global production strategy is a simple theoretical concept employee by the passing algorithm to perform insertion, deletion on processing tokens to recover from erroneous input. The advantage of this strategy is that it helps in improving the time and space requirement of parsing.

4.3.2 Context-Free Grammars

Q17. Explain in detail about Context-Free Grammar (CFG).

Answer :

For answer refer Unit-II, Page No. 50, Q.No. 29.

Q18. Discuss about the notational conventions of context free grammars.

Answer :

The notational conventions of context free grammars are listed below,

The following symbols are treated as non-terminals,

- (i) The initial uppercase letters of the alphabet. For instance, A, B, C.
- (ii) The italic words in lower case like *expr* or *stmt*.
- (iii) The start symbol of the grammar i.e., S.
- (iv) In case of programming constructs, uppercase letters denote the constructs. For instance, E represents expressions, T represents terms and F represents factors respectively,

The following symbols are treated as terminals,

- (i) The initial letters of the alphabet in lowercase like a, b, c.
- (ii) The digits 0 to 9 (0, 1, ..., 9).
- (iii) Strings in bold representing terminal symbols. For instance id, if.
- (iv) Operator like +, *, etc.
- (v) Punctuations like comma, semicolon, parenthesis.

The last letters of the alphabet in uppercase like x, y, z denote the grammar symbols. (either terminals or non terminals).

The Greek letters in lowercase like α , β , γ denote the strings of grammar symbols which may be empty.

This helps in writing the generic production as $A \rightarrow \alpha$, where A represents head of the production and α represents the body of the production.

The last letters of the alphabet in lowercase, (usually u, v, w, ..., z) denote strings of terminals. These letters may also be empty.

The head of the initial production is considered as the start symbol if the start symbol is not specified.

The productions of the form $A \rightarrow \alpha_1, A \rightarrow \alpha_2, A \rightarrow \alpha_3, \dots, A \rightarrow \alpha_k$ are called as A-productions because all these productions have A as their head. They can also be represented as $A \rightarrow \alpha_1 | \alpha_2 | \alpha_3 | \dots | \alpha_k$.

Where, $\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_k$ are the alternatives for A.

Q19. How the language generated by a grammar is verified? Explain with an example.

Answer :

The grammar G can generate a language L, iff the following conditions are satisfied,

- (i) Every string generated by grammar G must be present in language L.
- (ii) Every string present in the language L can be generated by the grammar G.

To illustrate the above cases consider the following examples,

Example

Consider a grammar

$$A \rightarrow (A)A \mid \epsilon$$

Which is capable of generating only strings of balanced parenthesis.

For the above grammar, the following statements are true,

- (a) Every string derived from A is a string with equal number of parenthesis.
- (b) Every string with equal number of parenthesis can be derived from A.

Proof for Case(a)

Basis Hypothesis

In a single step (i.e., $n = 1$) only empty string (ϵ) can be derived from A which is considered as balanced.

Induction Hypothesis

Suppose that strings which are derived in less than n steps are balanced. Also consider an n-step leftmost derivation. The derivation of both these considerations is in the following form,

$$A \xrightarrow{lm} (A)A \xrightarrow{lm} (x)A \xrightarrow{lm} (x)y$$

Proof

From the above derivation, it is clear that x and y derivations are carried out in less than n steps. Thus, by induction hypothesis x and y are balanced strings. Hence the string (x)y also needs to be balanced.

Proof for Case (b)

Basic Hypothesis

In case if the length of string is zero, the string must be ϵ which is a balanced string.

Induction Hypothesis

It is a fact that length of balance strings is even. Consider that a balanced strings having length less than $2n$ can be derived from A. Also consider, a string w which is a balanced string having length $2n$ where $n \geq 1$ and (x) as the smallest (nonempty) prefix of string w with balanced parenthesis.

From the above assumptions w can be defined as,

$$w = (x)y$$

Where x and y are balanced strings whose lengths are less than $2n$.

Therefore, it can be said that x and y can be derived from A . Thus, it is now clear that the derivation is of the form.

$$A \xRightarrow{lm} (A) A \xRightarrow{*} (x) A \xRightarrow{lm} (x)y$$

This means $w = (x)y$ can be derived from A .

Hence it is proved that a grammar G generates a language L .

Q20. Discuss how context free grammar is more powerful than Regular expression.

Answer :

Context-Free Grammars Versus Regular Expressions

Context-free Grammars (CFG) are powerful than regular expressions (RE). This is because of the following features of CFG,

- CFGs are capable of describing all the constructs that are described by RE. However, REs cannot describe all the constructs that can be described by CFGs.
- The languages which are regular languages are context free languages. However, the languages which CFLs are not necessarily be regular.

The above facts can be illustrated through the following example,

Consider a language consisting of a set of strings of a 's and b 's ending in abb as described by a RE $(a/b)^*abb$. The same language can be described by a CFG as,

$$S_0 \rightarrow aS_0 | bS_0 | aS_1$$

$$S_1 \rightarrow bS_2$$

$$S_2 \rightarrow bS_3$$

$$S_3 \rightarrow \epsilon$$

A grammar can be developed mechanically which can accept the language that is accepted by NFA. Thus, the NFA accepting the above languages is given below,

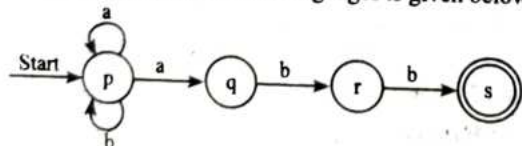


Figure: NFA Accepting Strings of a 's and b 's Ending in abb

Now, a CFG can be constructed as follows,

- A non-terminal S_i is created for every state i of the NFA.
- The production $S_i \rightarrow aS_j$ is introduced for the transition of state i to state j on input a .
- The production $S_i \rightarrow \epsilon$ is introduced if the accepting state is i .
- The start symbol is made as S_i for the start state i .

In addition, a language with equal number of x 's and y 's [i.e., $L = x^ny^n$ where $n \geq 1$] is a language which can be generated by context free grammars but not by the RE.

The reason why this language cannot be generated by RE is discussed below,

Assume that, the language $L = x^ny^n$ is generated by some regular expression. So it is possible to construct a DFA D accepting L with some k states. In case of input having more x 's than DFA states (i.e., k) the DFA D has to enter into some of the states (say A_i) for two times. Name the path, i.e., from state A_i to itself as a^i . Now, there must also be a path from A_i to an accepting state (f) (as $a^ib^i \in L$). Moreover, there exist a path from A_i through A_j to the accepting state which is named as a^jb^j . This means that L also accepts a^jb^i also which is not in the language (a^ib^i). Thus, the $L = a^ib^i$ is accepted by DFA D .

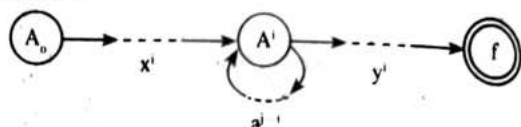


Figure: DFA Accepting x^iy^i and x^jb^j

Thus, it is concluded that the finite automata cannot accept the language such as $L = x^ny^n$. That demands the count of number of x 's before it completely gets y 's.

4.3.3 Writing a Grammar

Q21. Explain in detail about writing a grammar.

Answer :

A grammar can be rewritten in order to make it more suitable for parsing. The different techniques used for rewriting a grammar are as follows,

1. Eliminating Ambiguity

For answer refer Unit-II, Page No. 58, Q42, Topic: Removing Ambiguity from a Grammar.

2. Eliminating Left Recursion

Grammar is said to be left recursive, if the same non-terminal on the left hand side of a production appears as the first symbol in the right hand side. In other words if there is a production of the form,

$$A \Rightarrow A\alpha$$

For some string α then grammar is left recursive. So, eliminate left recursion since top-down parsers cannot handle left recursive grammars.

An immediate left recursion from the production of the form is,

$$A \rightarrow A\alpha \mid B$$

It can be eliminated by rewriting the productions as,

$$A \rightarrow \beta A'$$

$$A \rightarrow \alpha A' \mid \epsilon$$

which does not change the set of strings that can be derived from A .

Example

Consider the CFG for arithmetic expressions as,

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

The production for E and T are left recursive. After eliminating the left-recursions from these productions, the grammar can be obtained as,

$$E \rightarrow TE'$$

$$E' \rightarrow + TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow * FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

In general, if there are many productions for A then combine all A -productions as given below.

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$$

Where no β_i can start with an A

Then, replace them by following productions,

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$$

Left Factoring

Left factoring is a grammar transformation in which the common parts of two productions are isolated into a single production. A left factored grammar is suitable for predictive parsing.

Any production of the form, $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ (where α is common) can be replaced by following production as,

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

Left factoring is required because it is difficult to decide which production to choose either $\alpha\beta_1$ or $\alpha\beta_2$ to expand A when the input starts with a non-empty string derived from α . In left factoring, this decision can be deferred by expanding A to $\alpha A'$ until enough input is seen to make the right choice.

Example

Consider the following grammar,

$$R \rightarrow R' \mid R \mid RR \mid R^* \mid (R) \mid a \mid b$$

Here the productions $R \rightarrow R' \mid RR \mid R^*$ have R as the common part.

Hence, the above productions as can be written as follows,

$$R \rightarrow RS$$

$$S \rightarrow ' \mid * \mid (\mid) \mid a \mid b$$

Therefore, after eliminating left factoring, the following productions are obtained.

$$R \rightarrow RS \mid R' \mid (R) \mid a \mid b$$

$$S \rightarrow ' \mid * \mid (\mid) \mid a \mid b$$

Q22. Discuss in detail about non-context-free language construct.

Answer :

Some syntactic constructs of non-context-free programming languages are difficult to be specified using only grammars. These difficulties can be illustrated with the following examples.

Example-1

Consider the abstract language $L = \{wcw/w \text{ is in } (a/b)^*\}$. In this language, the first w is the identifier declaration, c is the piece of program code and second w is the inclusion of the identifier. The language 'L' consists of strings like aabbcaabb i.e., repeated number of a 's and b 's separated by ' c '. Such non-context-free language is directly related to non-context-free programming languages like C and Java. This is because, programming languages demand identifier declaration prior to its usage. Moreover, arbitrary length can be allotted to the identifiers.

The grammar of these programming languages represent identifiers using tokens like 'id' rather than distinguishing identifiers based on the type of character strings. The condition of declaration of identifiers before their usage is verified or checked by the semantic-analysis phase in the compilers.

Example-2

Consider a non-context-free language $L = \{a^n b^m c^n \mid n \geq 1 \text{ and } m \geq 1\}$. This language consists of strings, which are the output of regular expression $a^* b^* c^* d^*$.

Where;

Number of a 's and c 's = number of b 's and d 's.

In this language, a^n represents that the variable a can be repeated n number of times and a^* , b^* represents the formal parameter lists of two functions a and b having formal parameters n and m respectively. However, c^n and d^m represent the actual parameter lists involved in calling the functions.

This languages 'L' is capable of abstracting the difficulties involved in verifying that number of formal parameters in function declaration must coincide with the number of actual parameters in the usage of function.

In programming languages like C, the syntax of function declaration does not involve the count of parameters. Therefore, it is done using semantic-analysis phase.

4.3.4 Top-Down Parsing

Q23. What is top-down parsing? Explain different types of top-down parsing techniques.

Answer :

Top-down Parsing

Top-down parsing may be considered as an attempt to build a parse tree for an input string in preorder, that is starting from the root and then creating the nodes of a parse tree. It can also be considered as an attempt to construct a leftmost derivation for an input string.

A top-down parser constructs the leftmost derivation from the start symbol of the grammar. Then it selects a suitable production rule such that it can move the input string from left to right in the sentential form. If there are more than one production rules for a leftmost non-terminal then selection of production rule is dependent on whether the parse tree can backtrack or not. If the parser can backtrack, it can scan the input string repetitively and try out all the possibilities in any order until it has succeeded in parsing the string. On the other hand if the backtracking is not permitted then the parser has to make the right selection of the production rule, which is crucial task in such parsers.

Top-down parsers are popular since it is easy to construct efficient parsers by hand using top-down methods.

Types of Top-down Parsing

The different types of top-down parsing techniques are.

(i) Recursive Descent Parsing

For answer refer Unit-IV, Page No. 112, Q24.

(ii) Non-recursive Predictive Parsing

For answer refer Unit-IV, Page No. 116, Q28.

Q24. Discuss in detail Recursive-Descent parsing.

Answer :

Recursive Descent Parsing

Parsing means generating a parse tree. Now, in Top-down parsing a parse tree is generated by taking the parent node or the starting variable of the grammar and constructing the nodes in preorder.

The recursive descent parsing method is the general form or general way of representing top-down parsing. In this type of parser, the concept of repeated scans of input may be used. Making repeated scans or performing repeated reading of the input is called as backtracking.

Steps

Every non-terminal has a separate procedure. The procedure body of the corresponding non-terminal is referred to as its RHS. The RHS of the production rule is converted to a code by parsing symbol by symbol using a recursive descent parser.

The basic steps to be followed while constructing recursive descent parser are as follows,

1. If terminal is the input symbol then, it is matched with the lookahead symbol. If the match becomes successful then the lookahead pointer is incremented by 1 to point the next input symbol.
2. If non-terminal is the input symbol then, the procedure of the corresponding non-terminal is called.

3. If there are many alternatives for the production rule then, all the alternatives are combined to form a single procedure body.
4. Finally, the production corresponding to the start symbol activates the parser.

Lets take an example for recursive descent parsing, which needs backtracking.

Example

$$G = (\{I, B\}, \{a, b, c, d\}, R, \{I\})$$

R is given by

$$I \rightarrow cBd$$

$$B \rightarrow ab|a$$

and the input string that is to be parsed is cad.

As it is known that, to parse an input string, a syntax tree is constructed using the start variable and nodes are created from left to right, in preorder.

The parser has a pointer, pointing to the first symbol of the input string.

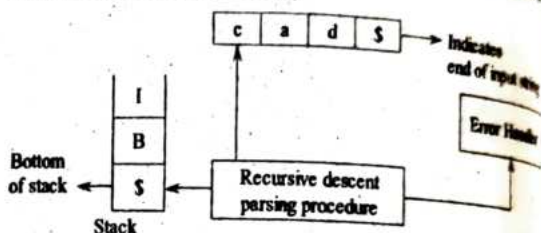
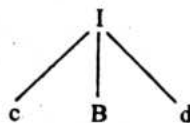


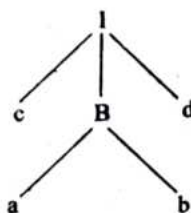
Figure: Structure of a Recursive Descent Parser

The parser starts parsing from the initial or start variable i.e., I and generates the syntax tree for the production $I \rightarrow cBd$.
yield of I

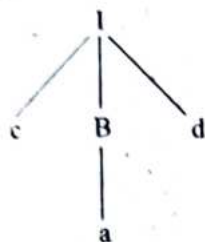


Now, as shown in the figure the parser checks the first input symbol i.e., 'c' and compares 'c' with the yield of I from left to right. As the first input 'c' matches the left most node 'c' in the syntax tree, the parser increments the pointer to next symbol. The parser is now pointing to the input symbol 'a', it compares a to the next node in the tree i.e., B , as B is a non terminal it expands B with its yield, if more than one yield exists then priority is given to the first yield from left.

$$B \rightarrow ab|a$$



Now, parser pointing to input symbol 'a' and the left most child of variable B is 'a' finds a match and the pointer is incremented to next input symbol i.e., 'd'. The next child of B is 'b' which does not matches with 'd'. At this stage, the parser does backtracking for the variable B . It checks that if has any more yields which can be used. If no more yields then the parser generates an error. If yes, then expands B or replaces B with that yields and constructs a new tree.



The input symbol 'a' now matches the child 'a' hence the parser then increments the pointer. Now, the parser is pointing to input symbol 'd'.

Now, the right most child of variable I matches the input symbol d .

Now, the parser halts and declares that the input string is syntactically correct.

There are few grammars that can cause a recursive descent parser to enter into an infinite loop. One of such grammars is known as left-recursive grammar.

Q25. Explain in detail first and follow functions. Give examples.

Answer :

FIRST Function

$FIRST(\alpha)$ is a set of terminal symbols that are the first symbols in the strings derived from α . Where α is any string of grammar symbols.

Rules for FIRST Function

The rules for computing $FIRST(X)$ for all grammar symbols X are,

1. If X is a terminal symbol then $FIRST(X) = \{X\}$
2. If X is a nonterminal having a production rule as, $X \rightarrow \epsilon$ then $FIRST(X) = \{\epsilon\}$.
3. If X is a non-terminal having a production rule as,
 $X \rightarrow A_1 A_2 \dots A_n$ then add a $FIRST(X)$ if for some i , there is,
 - (a) a in $FIRST(A_i)$ and
 - (b) ϵ in $FIRST$ of all A_i i.e., $FIRST(A_1), \dots, FIRST(A_{i-1})$

If $FIRST(A_j)$ for all $j = 1, 2, \dots, k$ is ϵ then ϵ is added to $FIRST(X)$.

That is whatever in $FIRST(A_j)$ is in $FIRST(X)$. If ϵ is not in $FIRST(A_j)$ then nothing more is added to $FIRST(X)$.

However, if $FIRST(A_j)$ derives ϵ in zero or more steps then, add $FIRST(A_j)$, $FIRST(A_j)$ and so on.

The above rules for $FIRST(X)$ are applied until there are no more terminals or ϵ that can be added to any $FIRST$ set.

FOLLOW Function

$FOLLOW(A)$ is a set of terminal symbols as such that they appear immediately after A in any string occurring on the right hand side of production.

Rules for FOLLOW Function

The rules for computing $FOLLOW(A)$ for all non-terminals A of the grammar are,

1. For the start symbol S , $FOLLOW(S) = \{\$ \}$ where $\$$ is used as end marker for the input.
2. If A has a production as $A \rightarrow \alpha B \beta$. Where $FIRST(\beta)$ does not contain ϵ then $FOLLOW(B) = FIRST(\beta) - \{\epsilon\}$.
3. If A has a production as $A \rightarrow \alpha B \beta$. Where $FIRST(\beta)$ contain ϵ then $FOLLOW(B) = FIRST(\beta) - \{\epsilon\} \cup FOLLOW(A)$.
4. If A has a production as $A \rightarrow \alpha B$ then $FOLLOW(B) = FOLLOW(A)$.
5. If A is the rightmost symbol in some sentential form then S is added to $FOLLOW(A)$.

Example

Consider the grammar given below,

$G: S \rightarrow aAB$

$S \rightarrow bA$

$S \rightarrow \epsilon$

$A \rightarrow aAb$

$A \rightarrow \epsilon$

$B \rightarrow bB$

$B \rightarrow \epsilon$

The above grammar (G) is free from left recursion and left factor.

Computing FIRST and FOLLOW

FIRST

$$FIRST(S) = FIRST(a) \cup FIRST(b) \cup FIRST(\epsilon) \quad (\text{using rule 1 and 2})$$

$$= \{a, b, \epsilon\}$$

$$FIRST(A) = FIRST(a) \cup FIRST(\epsilon) \quad (\text{using rule 1 and 2})$$

$$= \{a, \epsilon\}$$

$$FIRST(B) = FIRST(b) \cup FIRST(\epsilon) \quad (\text{using rule 1 and 2})$$

$$= \{b, \epsilon\}$$

Therefore first of S , A and B are as follows,

$$\text{FIRST}(S) = \{a, b, \epsilon\}$$

$$\text{FIRST}(A) = \{a, \epsilon\}$$

$$\text{FIRST}(B) = \{b, \epsilon\}$$

FOLLOW(A)

$$\text{FOLLOW}(S) = \{\$ \} \text{ (As } S \text{ is the start symbol)}$$

$$\text{FOLLOW}(A)$$

The production involving $A \rightarrow aA$ is of form $A \rightarrow \alpha B \beta$ using rule (2) for FOLLOW is given below,

$$S \rightarrow aAB$$

$$A \rightarrow \alpha B \beta$$

$$\text{Then FIRST}(\beta) = \epsilon$$

$$= \text{FIRST}(B) = \epsilon$$

$$= \{b, \epsilon\} = \epsilon$$

$$= \{b\}$$

Consider the production $S \rightarrow bA$

The production involving A is $S \rightarrow bA$

$$\text{FOLLOW}(A) = \text{FOLLOW}(B)$$

$$\text{FOLLOW}(A) = \text{FOLLOW}(S)$$

$$= \{\$ \}$$

$$\therefore \text{FOLLOW}(A) = \{b, \$ \}$$

FOLLOW(B)

The production involving B is $S \rightarrow aAB$ is of form $A \rightarrow \alpha B \beta$ using rule (3) for FOLLOW is given below,

$$S \rightarrow aAB$$

$$A \rightarrow \alpha B \beta$$

$$\text{Hence } \beta = \epsilon$$

Hence, by using rule (3) the FOLLOW's of B is obtained,

$$\text{FOLLOW}(B) = \text{FOLLOW}(S)$$

$$= \{\$ \}$$

$$\therefore \text{FOLLOW}(B) = \{\$ \}$$

Hence, the FOLLOWs of A and B are given below,

$$\text{FOLLOW}(A) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{b, \$ \}$$

$$\text{FOLLOW}(B) = \{\$ \}$$

Q26. What are LL(1) grammars? Explain in detail.

OR

Test whether the grammar is LL(1) or not and construct a predictive parsing table for it $S \rightarrow iEtS | iEtSeS | a$, $E \rightarrow b$.

Answer :

LL(1) Grammar

A grammar is said to be LL(1) grammar if its parsing table does not contain multiply-defined entries that is an entry $M[A, a]$ always contain a single production $A \rightarrow \alpha$ from the grammar.

Here,

L : Stands for left to right scanning of the input.

L : Stands for producing the leftmost derivation for the input.

1 : Stands for using one look ahead symbol for making parsing action decisions.

LL(1) grammar has the following properties.

1. A grammar which is ambiguous or left recursive, cannot be a LL(1) grammar.
2. The parsing table of an LL(1) grammar does not have multiply-defined entries.
3. If $A \rightarrow \alpha \mid \beta$ are two distinct productions, then LL(1) grammar must hold the following conditions
 - (i) The strings derived from α and β should not begin with the same terminal a i.e., $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) \neq a$.
 - (ii) Either α or β can derive the empty string i.e., $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) \neq \epsilon$.
 - (iii) If β derives an empty string then α should not derive strings that begins with the terminals in $\text{FOLLOW}(A)$ i.e., $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \phi$.

The same argument holds if α derives an empty string.

ALL(1) parser provides the detailed information on the internal of the parsing process. It is also capable of catching all errors because the situations of syntax errors are recorded explicitly in the table.

An example of a LL(1) grammar is a grammar for arithmetic expressions. A non LL(1) grammars have multiply-defined entries in their parsing table.

Example

Consider the below dangling else grammar.

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

$$E \rightarrow b$$

On left factoring the above grammar, the following productions are obtained.

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

The parsing table for this grammar is given below.

Non-terminal	Input Symbol					
	a	b	e	i	t	\$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Table: Parsing Table for Dangling-else Grammar

In the above parsing table the entry $M[S' e]$ defines multiple entries for the production S' .

This grammar is ambiguous because it is not clear which one of the productions $S' \rightarrow \epsilon$ or $S' \rightarrow eS$ to be chosen upon seeing the input symbol e .

If the parser chooses $S' \rightarrow \epsilon$ then this prevents e from ever being put on the stack or removed from the stack. If the second choice $S' \rightarrow eS$ is made then it resolves the ambiguity. This choice associates the else's with the closest previous then's.

So the parser can parse the string by making the multiple defined entry as a single valued entry i.e., $M[S', e] =$

The multiple-defined entries from a parsing table can be removed by first eliminating the left recursion and factoring the grammar then constructing a parsing table for the transformed grammar.

Q27. Construct LL(1) parsing table for the following grammar,

$S \rightarrow CC$

$C \rightarrow cC \mid d$, where S is start variable, $V = \{S, C\}$, $T = \{c, d\}$.

Write and apply LL(1) parsing algorithm to parse the string $cdcd$.

Answer :

Given grammar G is,

$G: S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$

The above grammar (G) is free from left recursion and left factor.

Computing **FIRST()** and **FOLLOW()**

FIRST()

$\text{FIRST}(S) = \text{FIRST}(C) = \{c, d\}$

FOLLOW()

$\text{FOLLOW}(S) = \{\$ \}$ [As S is the start symbol]

$\text{FOLLOW}(C)$

The production involving ' C ' are,

$S \rightarrow CC$

$C \rightarrow cC$

$\text{FOLLOW}(C) = \text{First}(C)$
 $= \{c, d\}$

Hence,

$\text{FOLLOW}(S) = \{\$ \}$

$\text{FOLLOW}(C) = \{c, d, \$ \}$

Parsing Table

Variables	c	d	\$
S	$S \rightarrow CC$	$S \rightarrow CC$	$S \rightarrow CC$
C	$C \rightarrow cC$	$C \rightarrow d$	

Q28. Explain the procedure for non-recursive predictive parsing table with an example.

Answer :

Non-recursive Predictive Parsing

A non-recursive predictive parser uses a parsing table that shows which production rule to select from several alternatives available for expanding a given non-terminal and the first terminal symbol that should be produced by that non-terminal. The parsing table consists of rows and columns where there is a row for each non-terminal and a column for each terminal symbol including \$, the end marker for the input string. Each entry $M[A, a]$ in a table is either a production rule or an error. It also uses a stack containing a sequence of grammar symbols with \$, symbol placed on the bottom indicating the bottom of the stack. Initially, the start symbol resides on top. The stack is used to keep track of all the nonterminals for which no prediction has been made yet.

The parser also uses an input buffer and an output stream. The string to be parsed is stored in the input buffer. The end of the buffer uses a \$ symbol to indicate the end of the input string.

The configuration of the non-recursive predictive parser is shown in the figure.

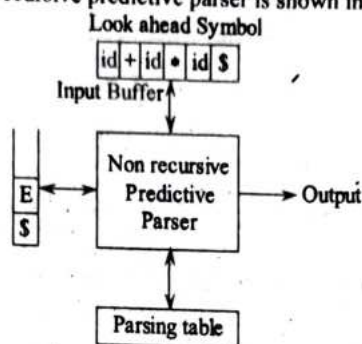


Figure: Non-recursive Predictive Parser Configuration

The program for non-recursive predictive parser acts on the basis of two symbols X and a . Where X is the grammar symbol on top of the stack and a is the current lookahead symbol in the input buffer. These symbols together define the actions of the parser is given as,

1. If the top of the stack is \$ and the lookahead symbol is also \$ i.e., $X = a = \$$ then it means parser has parsed the string successfully and the parser halts.
2. If the top of the stack is terminal which is same as the current lookahead symbol i.e., $X = a \neq \$$ then, parser pops off X from the stack and advances the lookahead symbol.
3. If the top of the stack is a non-terminal then the parser checks the entry $M[X, a]$ for this nonterminal in the parsing table. The entry $M[X, a]$ may contain a production rule or an error. If this entry contains a production as $X \rightarrow LMN$ then parser replaces the non-terminal X on the stack with the right hand side of this rule in the reverse order i.e., N being on top of stack. If this entry contains an error then the parser calls an error recovery routine. The parser prints the production rules used by it as the output.

Example: Consider the grammar for arithmetic expressions as,

$$E \rightarrow TE'$$

$$E' \rightarrow + TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow * FT' \mid \epsilon$$

$$F \rightarrow (\epsilon) \mid id$$

The parsing table for this grammar is shown in table. Where the empty entries are error entries. The filled in entries show production rules which must be used for expanding non-terminals on top of the stack.

Non-terminal	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (\epsilon)$		

Table: Recursive Descent Parsing Table

The sequence of moves made by the parser while parsing the input $id + id * id$ are shown below. In the INPUT column, the lookahead is the leftmost symbol of the string. The OUTPUT column shows the productions used by parser for expanding the nonterminal on the top of the stack. If the output is observed carefully, then, it can be noticed that the parser is tracing out a leftmost derivation for the input.

STACK	INPUT	OUTPUT
\$	$id + id * id \$$	
SET	$id + id * id \$$	$E \rightarrow TE'$
$SETT$	$id + id * id \$$	$T \rightarrow FT'$
$SETTid$	$id + id * id \$$	$F \rightarrow id$
$SETT'$	$+ id * id \$$	id is matched
SET	$+ id * id \$$	$T' \rightarrow \epsilon$
$SETT$	$+ id * id \$$	$E' \rightarrow + TE'$
SET	$id * id \$$	$+$ is matched
$SETT$	$id * id \$$	$T \rightarrow FT'$
$SETTid$	$id * id \$$	$F \rightarrow id$
$SETT'$	$* id \$$	id is matched
$SETTF$	$* id \$$	$T' \rightarrow * FT'$
$SETTF$	$id \$$	$*$ is matched
$SETTid$	$id \$$	$F' \rightarrow id$
SET	$\$$	id is matched
SET'	$\$$	$T \rightarrow \epsilon$
S	$\$$	$E \rightarrow \epsilon$
$\$$ is matched and processor halts with a successful parsing of input.		

Table: Parsing of Input $id + id * id$ by Predictive Parser

Q29. Discuss about error recovery in predictive parsing.

Answer :

Error Recovery in LL Parsing

A predictive parser is said to be an LL parser if in the predictive parsing table there exist no multiple defined entries.

Cause of Error

All parser detects an error whenever the terminal on top of stack do not match the current input symbol. And when the top of stack symbol is a nonterminal which do not match the current input symbol, the entry in the table is given by,

$$T[X, a] = \text{Error}$$

Where, T is the parsing table,

X is a nonterminal and

' a ' is the current input symbol.

- ❖ The LL parser when encounters an error can either generate a diagnostic error message to the user or try to recover from error. The generation of a diagnostic message should be user friendly and it is used only when the parser is not sure enough to do proper modification to the input (without changing the meaning of input).

Error recovery done by a LL parser can be either of the following,

1. Panic-mode error recovery
2. Phrase-level error recovery.

1. Panic-mode Error Recovery

The idea behind panic-mode error recovery is skipping the input symbols until a token is identified that belongs to the set of synchronizing tokens. Thus, the selection of synchronizing tokens would imply the effectiveness of this error recovery method.

- ❖ The functions $\text{first}()$ and $\text{follow}()$ are used to compute the synchronizing tokens set. However, ϵ -productions are also used as one more way to construct synchronizing tokens set.

(i) $\text{follow}()$ Used to Compute SS (Synchronizing tokens set).

All the symbols in $\text{follow}(B)$ are placed into SS for a nonterminal ' B '. That means, whenever an error is encountered, the input symbols are skipped until a symbol from $\text{follow}(B)$ is seen and then $\text{pop } B$ from stack, which may help the parser in continuing parsing.

- ❖ However, using $\text{follow}(B)$ to be the SS for B is not enough to continue parsing whenever an error has occurred. This is because, in language C every statement ends with a semicolon, and keywords are those tokens that appear at the beginning of a statement may not fall in $\text{follow}(E)$, where E is a non-terminal that generates expressions. Thus, a missing semicolon after an assignment will lead in skipping of a keyword appearing in the next statement.

- ❖ There is a method or way to solve the problems related to keyword skipping. This method says add to the SS of a lower constructs the symbols that begin the higher constructs. For examples, an expression comes in a statement and a statement comes in a block and so on.

(ii) Using first() to Compute SS

All the symbols in first(B) are placed in SS for a nonterminal B . That means, whenever an error is encountered, the input symbols are skipped until a symbol is seen or identify from first(B) and then pop B from stack.

(iii) Using ϵ -productions to Compute SS

This method says if a nonterminal generates or produces an empty string ' ϵ ' and there is no entry between the nonterminal and input symbol then use $A \rightarrow \epsilon$ as the default method to continue parsing. However, this method causes the error detection to be postponed, but will never cause an error to be missed. Hence, the method of using ϵ -productions minimizes the number of nonterminals that needs to be considered during error recovery.

Now, when the parser is not able to continue parsing even after using any of the methods above, the parser will simply pop the terminal (which is not matching to the top of the symbol) out of stack. After popping the terminal the parser will display or generate an error message about the same.

2. Phrase Level Error Recovery

Phrase-level error recovery means filling the blank or empty entries in the parsing table with pointers to routines that are described in error handler part of a compiler. The routines may do a number of modifications on stack as well as on input symbols. The operations that these routines may perform are inserting, deleting the input symbol and popping the top of stack symbol, pushing new symbols on stack.

- ❖ If the above method is used then user should be careful that the routines do not make the parser not to accept even a single word of the language. One more thing where user need to be caution is that the error routines do not enter into an infinite loop.

Q30. Construct a predictive parsing table for the Grammar

$E \rightarrow E+T \mid T, T \rightarrow T^*F \mid F, F \rightarrow (E) \mid id.$

Answer :

Given grammar is,

$E \rightarrow E+T \mid T$

$T \rightarrow T^*F \mid F$

$F \rightarrow (E) \mid id$

On eliminating Left recursion from the above grammar, the following productions are obtained.

$G: E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

Computing first() and follow()

$\text{first}(+) = \{+\}$

$\text{first}(*) = \{*\}$

$\text{first}(\$) = \{\$\}$

$\text{first}(\epsilon) = \{\epsilon\}$

$\text{first}(id) = \{id\}$

$\text{first}(\) = \{(\}$

$\text{first}(\) = \{)\}$

$\text{first}(E) = \text{first}(T) = \text{first}(F) = \{(\ , id\}$

$\text{first}(E') = \{+, \epsilon\}$

$\text{first}(T') = \{*, \epsilon\}$

$\text{follow}(E) = \{), \$\}$

$\text{follow}(E') = \{), \$\}$

$\text{follow}(T) = \{+,), \$\}$

$\text{follow}(T') = \{+,), \$\}$

$\text{follow}(F) = \{+, *,), \$\}$

Following the procedure for constructing predictive parsing table,

$E \rightarrow TE'$ is of the form $I \rightarrow \beta$

where, $I = E$ and $\beta = TE'$

$\text{first}(\beta) = \text{first}(TE') = \text{first}(T) = \{(\ , id\}$

$\therefore$ The production $E \rightarrow TE'$ is placed in $T[E, (]$ and $T[E, id]$

in $T[e, (]$ and $T[E, id]$

$E' \rightarrow +TE'$ is of the form $I \rightarrow \beta$

where, $I = E'$, $\beta = +TE'$

$\text{first}(\beta) = \text{first}(+TE') = \text{first}(+) = \{+\}$

Therefore, the production $E' \rightarrow +TE'$ is placed in $T[E', +]$.

Similarly, $E' \rightarrow \epsilon$

$(I \rightarrow \beta)$

$I = E'$

$\beta = \epsilon$

$\text{first}(\beta) = \text{first}(\epsilon) = \{\epsilon\}$

$[\therefore \text{follow}(E') = \{), \$\}]$

$\therefore$ The production $E' \rightarrow \epsilon$ is placed in $T[E',)]$ and $T[E', \$]$

$T \rightarrow FT'$

$(I \rightarrow \beta)$

$I = T$

$\beta = FT'$

$\text{first}(FT') = \text{first}(F) = \{(\ , id\}$

$\therefore$ The production $T \rightarrow FT'$ is placed in $T[T, (]$ and $T[T, id]$

$T' \rightarrow *FT'$

$(I \rightarrow \beta)$

$I = T'$

$\beta = *FT'$

$\text{first}(\beta) = \text{first}(*FT') = \{*\}$

The production $T' \rightarrow *FT'$ is placed in $T[T', *]$

$$T' \rightarrow \epsilon$$

$$(I \rightarrow \beta)$$

$$I = T'$$

$$\beta = \epsilon$$

$$\text{first}(\epsilon) = \{\epsilon\}$$

$$\text{follow}(T') = \{+,), \$\}$$

The production $T' \rightarrow \epsilon$ is placed in $T[T', +]$, $T[(T',)]$ and $T[T', \$]$

$$F \rightarrow (E)$$

$$(I \rightarrow \beta)$$

$$I = F$$

$$\beta = (E)$$

$$\text{first}(\beta) = \text{first}((E)) = \{()\}$$

The production $F \rightarrow (E)$ is placed in $T[F, (]$

$$F \rightarrow id$$

$$(I \rightarrow \beta)$$

$$I = F$$

$$\beta = id$$

$$\text{first}(\beta) = \text{first}(id) = \{id\}$$

The production $F \rightarrow id$ is placed in $T[F, id]$

The predictive parsing table for the given grammar is shown in table below.

Variable	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Table: Predictive Parsing Table for G

4.3.5 Bottom-up Parsing

Q31. What is bottom-up parsing and reductions? Explain with example.

Answer :

Bottom-up Parsing

Bottom-up parsing is a parsing terminology that attempts to construct a parse tree by starting from the leaves and reaching to the root. That is, it constructs the parse tree in bottom-up process. In other words it constructs the rightmost derivation of the input string in the reverse order by reducing the input string to the start symbol of the grammar. This process is referred to as reduction.

A bottom-up parser constructs the rightmost derivation of the input by matching the sub-string of input with any right-hand side of a production of the grammar. If this is matched then replace the string with the left-hand side of that production. The process of replacing right-hand side of a production by the left-hand side is called reduction. If the input string is reduced to the start symbol of the grammar then the parser is successful for parsing an input, otherwise it gives an error.

Example

Consider the following grammar,

$$S \rightarrow aABe$$

$$A \rightarrow Abc \mid b$$

$$B \rightarrow d$$

Now, consider the reduction of input string $abde$ to the start symbol ' S ' as in the above grammar. It involves following steps.

$$\begin{aligned} & abde \\ \Rightarrow & aAde \\ \Rightarrow & aABe \\ \Rightarrow & S \end{aligned}$$

Where, S is the start symbol of the grammar.

From the above example, it can be observed that the parser traces the rightmost derivation of the input string in the reverse order.

In the process of reduction one must identify a substring called 'handle' which matches the right-hand side of a production and when it is reduced to the left-hand side nonterminal leads to generation of previous right most sequential form in the right most derivation. The right-most derivation of some input string w is as follows.

$$S \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_{n-1} \Rightarrow \alpha_n$$

Then the handle within α_n must be identified in such a way that, by reducing α_n , the α_{n-1} can be obtained. That is, the previous right-most sequential form.

Therefore, in bottom-up parsing, identification of the handles to use them in reductions is an important task.

Q32. What is handle pruning? Explain in detail.

Answer :

Handle Pruning

A handle in a parse tree is a complete leftmost subtree that consists of interior node and its children. The act of removing the children of nonterminal A from the subtree is called as pruning the handle. It is required to obtain the leftmost derivation in the reverse order.

If w is a string of some grammar G , then the rightmost derivation of w is,

$$S \Rightarrow \alpha_0 \Rightarrow \alpha_1 \Rightarrow \dots \Rightarrow \alpha_{n-1} \Rightarrow \alpha_n$$

Where α_n is nothing but the string w . The rightmost derivation in reverse order i.e., α_n through S is constructed by first selecting a handle β_n within α_n and then replacing β_n with the nonterminal A_n (if there is a production as $A_n \rightarrow \beta_n$) to produce the previous right sentential form α_{n-1} . This process is repeated until the input string is successfully reduced to the start symbol of the grammar. The sequence of productions applied in reverse order is the rightmost derivation for the input string.

Consider the grammar for arithmetic expressions,

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow id$$

The rightmost derivation in reverse for the string $id_1 + id_2 * id_3$ is obtained as follows. Here the handle is underlined.

$$id_1 + id_2 * id_3 \quad (\text{use } E \rightarrow id)$$

$$E + id_2 * id_3 \quad (\text{use } E \rightarrow id)$$

$$E + E * id_3 \quad (\text{use } E \rightarrow id)$$

$$E + E * E \quad (\text{use } E \rightarrow E * E)$$

$$E + E \quad (\text{use } E \rightarrow E + E)$$

E is the start symbol of the grammar.

Q33. What is a shift reduce parser? Mention the conflicts that occur during shift-reduce parsing.

Answer :

Shift Reduce Parsers

Shift-reduce parser uses the bottom-up parsing technique to construct a syntax tree for an input string W . The construction of the syntax for an input string W of a grammar G begins from the leaf nodes and ends with the starting node or root node. A shift-reduce parser uses 4 functions while parsing a string, they are,

- (i) **Shift:** The action shift means shifting or pushing the next input symbol on the top of the stack.
- (ii) **Reduce:** Reduction means, replacing the right side of a production with its left side.
- (iii) **Accept:** The parser uses accept function whenever it is successful to parse the input string, i.e., reaching the root node in a parse tree.
- (iv) **Error:** This is used whenever a parser encounter an error during parsing of a string. This in turn invokes the error recovery procedure.

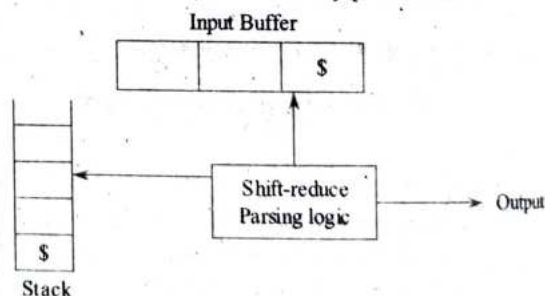


Figure: Shift Reduce Parser

Conflicts in Shift-reduce Parser

A shift-reduce parser has two conflicts. They are,

1. Shift/reduce conflict
2. Reduce/reduce conflict.

For the below grammar,

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

There exist both shift-reduce and reduce/reduce conflicts. This is explained below.

When the input buffer and stack looks like,

Stack Contents	Input Buffer
E^*E	+id \$
↑	↑

The parser encounters shift 'r' or reduce E^*E by E conflict. However, it gives preference to shift.

When the stack and input buffer looks like,

Stack Contents	Input Buffer
E^*E+E	\$
↑	↑

The parser is confused either to replace or reduce E^*E by E or $E+E$ by E . However, as the parser is using rightmost derivation in reverse order, the parser gives preference to $E+E$ reduction.

It can be said that, shift/reduce or reduce/reduce conflict will be encountered for those grammars which are not LR or they are ambiguous. However, the grammars that can be used in compiling are LR grammars. Thus, a shift-reduce parser cannot be constructed for a non LR grammar or ambiguous grammar. The limitation in a shift-reduce parser are overcome in other types of bottom-up parsers.

Q34. Differentiate between top-down and bottom-up parsing techniques.

Answer :

Top-down Parsing	Bottom-up Parsing
(i) The parsing of a string starts from root and ends by reaching the leaves.	(i) The parsing of a string starts from the leave nodes and ends with the root node.
(ii) Backtracking is implemented in one of the top-down parsers.	(ii) Backtracking cannot be implemented in bottom-up parsing.
(iii) There exist three types of top-down parsers : 1. Recursive-descent parser 2. Non-recursive predictive parser 3. Predictive parser.	(iii) There exist broadly three types of bottom-up parsers 1. Shift-reduce parser 2. Precedence parser 3. LR parsers. However, the 2 and 3 type of parsers are further divides as, ❖ Simple, operator precedence ❖ SLR, CLR, LALR parser.
(iv) Most of these parsers uses unambiguous and non-recursive (left) grammars.	(iv) These parser uses any type of grammar (can be even ambiguous).

4.3.6 Introduction to LR Parsing: Simple LR

Q35. What is LR parsing? Also discuss its advantages and disadvantages.

Answer :

LR Parsing

A bottom-up parsing technique called LR(k) parsing is used to parse a large class of context free grammars, where L stands for scanning of the input from left to right. R stands for creating a right most derivation in reverse order and k indicates the number of look ahead symbols used to make parsing decision. The LR(1) parsing can also be written as LR. LR parsers are of following types,

1. Simple LR (SLR)
2. Canonical LR (CLR)
3. Look Ahead LR (LALR)

The relationship among these parsers is expressed as, $SLR(1) \leq LALR(1) \leq CLR(1)$

Where the right-hand side parser parses a larger class of grammars than the left-hand side parser.

The structure of all these grammar is same i.e., they all are table drive except the parsing table.

Advantages

- (i) LR parsers can be constructed for those languages from which CFG's can be derived.
- (ii) LR parsing is the general shift-reduce parsing and non-backtracking technique.
- (iii) LR parsers are subset of those grammars which can be parsed by predictive parsers.
- (iv) Error detection in LR parsing is very quick.

Disadvantages

- (i) The main limitation of LR parsing is the manual creation of tables. It is very difficult as it requires more work to be done for a given grammar. Thus, leading to the need of automated LR passer.
- (ii) Simple LR method of LR parsing is not much efficient compared to other parsing methods.
- (iii) Canonical LR method of LR parsing is costlier and tedious.
- (iv) LALR parsing method of LR parsing suffer with reduce-conflict for some of the grammar.

Q36. Discuss in detail about items and LR(0) automaton.**Answer :****LR(0) Item or Items**

An LR (0) item for a grammar G is defined as a production rule $A \rightarrow \alpha$ of G with a dot inserted somewhere in the right-hand side of the rule.

For example, the possible LR(0) items for a production $A \rightarrow BCD$ are,

$$A \rightarrow \bullet BCD$$

$$A \rightarrow B \bullet CD$$

$$A \rightarrow BC \bullet D$$

$$A \rightarrow BCD$$

And for production $A \rightarrow \epsilon$, LR(0) item.

At any given point of parsing process, LR(0) item indicates how much portion of a production is visible.

A collection of sets of LR(0) items is called **canonical LR(0) collection** which is used in the construction of SLR parsers. LR(0) defines *augmented grammar* G' and two functions *closure* and *goto* in order to construct canonical LR(0) collection for a grammar G .

An augmented grammar G' for a grammar G , is G with a new start symbol S' defined using a production as $S' \rightarrow S \bullet$. The string is accepted by the parser if and only if the reduction $S' \rightarrow S$ takes place.

Closure Function

Let G be the grammar and I be the set of items of it then closure (I) is defined as the set of items constructed from I as follows,

1. Initially closure (I) contains every item in I .
2. If closure (I) contains an item $A \rightarrow \alpha \bullet B \beta$ and there is a production $B \rightarrow \gamma$ then the item $B \rightarrow \gamma$ is added to I , if it does not exist already.
3. Repeat step 2 until there are no new items that can be added to closure.

LR (0) Automaton

LR(0) automaton can be defined as the deterministic finite automaton which is constructed through canonical LR(0) collection. Every state of this automaton denotes a set of items ($I_1, I_2, \dots, I_n$) of canonical LR(0) collection.

Example

Consider an augmented grammar G' for arithmetic expression,

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Let I contains the set of one item $\{[E' \rightarrow \bullet E]\}$.

Now closure (I) is given by,

$$E' \rightarrow \bullet E$$

$$E \rightarrow \bullet E + T$$

$$E \rightarrow \bullet T$$

$$T \rightarrow \bullet T * F$$

$$T \rightarrow \bullet F$$

$$F \rightarrow \bullet (E)$$

$$F \rightarrow \bullet id$$

According to the rule (1), add $E' \rightarrow \bullet E$ to closure (I). Since, there is a nonterminal E to the right side of dot. So, add all the productions of E i.e., $E \rightarrow \bullet E + T$ and $E \rightarrow \bullet T$ to closure (I) by rule (2). Similarly, there is a nonterminal T to the right of dot so, add $T \rightarrow \bullet T * F$ and $T \rightarrow \bullet F$ productions of T to closure (I) again by rule (2). However, again there is F to the right of dot so, add the $F \rightarrow \bullet (E)$ and $F \rightarrow \bullet id$ productions of F to closure (I). Now no more terminals can be added to closure (I).

Goto Function

If I is a set of items of a grammar G and X is a grammar symbol then goto (I, X) is defined as the closure operation on the set of all items $A \rightarrow \alpha X \bullet \beta$ such that there is an item $A \rightarrow \alpha X \beta$ in I i.e.,

$$\text{goto}(A \rightarrow \alpha X \beta, X) = \text{Closure}(A \rightarrow \alpha X \bullet \beta)$$

Example

For the above grammar G' , if I contains the set of two items $\{[E' \rightarrow \bullet E], [E \rightarrow E \bullet + T]\}$. Then goto ($I, +$) consists of closure ($E \rightarrow E \bullet + T$) which is as follows,

$$E \rightarrow E \bullet + T$$

$$T \rightarrow \bullet T * F$$

$$T \rightarrow \bullet F$$

$$F \rightarrow \bullet (E)$$

$$F \rightarrow \bullet id$$

Here $E' \rightarrow E$ is ignored since there is symbol immediately to the right of the dot.

Construction of Canonical Collection of LR(0) Items

' C ' the canonical collection of LR(0) items for an augmented grammar G' is constructed using closure and goto functions as follows,

1. Initially ' C ' contains the closure ($S' \rightarrow \bullet S$).
2. For each set of items I present in C and each grammar symbol X do step 3.
3. If goto (I, X) is not empty and it is not present in C then add goto (I, X) to C .
4. Repeat step 3 until there are no more sets of items that can be added to C .

Example

The canonical collection of LR(0) items for the augmented expression grammar G' is given below.

- $I_0: E' \rightarrow \bullet E$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow id$
- $I_1: E' \rightarrow E \bullet$
 $E \rightarrow E \bullet + T$
- $I_2: E \rightarrow T \bullet$
 $T \rightarrow T \bullet * F$
- $I_3: T \rightarrow F \bullet$
- $I_4: F \rightarrow (\bullet E)$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow id$
- $I_5: F \rightarrow id \bullet$
- $I_6: E \rightarrow E + \bullet T$
 $T \rightarrow \bullet T * F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow id$
- $I_7: T \rightarrow T * \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow id$
- $I_8: F \rightarrow (E \bullet)$
 $E \rightarrow E \bullet + T$
- $I_9: E \rightarrow E + T \bullet$
 $T \rightarrow T \bullet * F$
- $I_{10}: T \rightarrow T * F \bullet$
- $I_{11}: F(E) \bullet$

The figure below shows the representation of goto function for this set of items as the transition diagram of a deterministic finite automata.

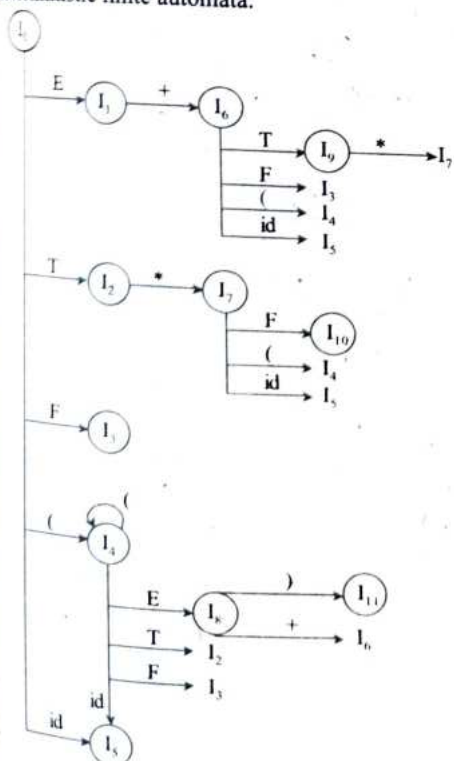


Figure: LR(0) Automaton

In the above grammar if I_0 is the only initial state and all other states are final states then the transition diagram recognizes exactly the viable prefixes of the given arithmetic expression grammar. A *viable prefix* is a prefix of a right sentential that can appear on top of the stack of a shift-reduce parser.

Q37. Discuss about,

- Model of an LR parser**
- Behavior of the LR parser.**

Answer :

(a) Model of an LR Parser

The LR parsers have the following structure. They consist of a stack, an input buffer, an output stream, a driven program and a parsing table consisting of two columns, action and goto.

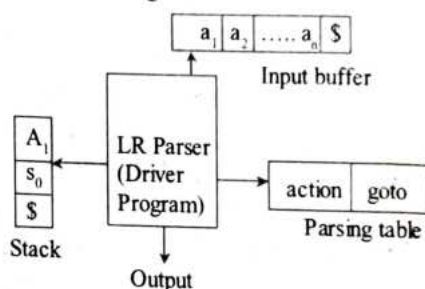


Figure: Structure of LR Parsers

All LR parsers have the same driver program but the contents of the parsing table changes. The stack contains *states* in addition to the grammar symbols. The contents of stack are $s_0 A_1 s_1 A_2 s_2 \dots A_m s_m$ where A is the grammar symbol and s_i is a state symbol with s_0 on the bottom and s_m on the top of the stack. The driver program takes the current input a_i from the input buffer and the state symbol on top of the stack to access the corresponding entry in the parsing table to determine the shift reduce decisions.

The parsing table consists of two parts, action and goto.

The entry action $[s_m, a_i]$ in the action column contains one of the following values,

- Shift to state s_i .
- Reduce by production $A \rightarrow \alpha$ of the grammar.
- Accept the input.
- Error entry.

The arguments to goto function is a state s_i on top of the stack and a grammar symbol A and this function returns a new state s_j .

(b) Behavior of the LR Parser

The LR parser driver program initializes the stack with s_0 and input buffer contains the input string as $w\$$. The driver program takes the state s_m on top of the stack and the current input symbol a_i and consult the parsing action table entry action $[s_m, a_i]$ for state s_m and input a_i and performs one of the moves depending upon the resulted action.

1. If action $[s_m, a_i] = s_r$, it pushes a_i then s_r on top of the stack and advances the pointer to next input symbol and continues.
2. If action $[s_m, a_i] = r$, i.e., reduce by using $A \rightarrow \alpha$, it pops $2r$ symbol from the stack where $r = \text{length}(\alpha)$. And pushes A then goto $[s_r, A]$ on top of the stack and continues.
3. If action $[s_m, a_i] = \text{accept}$, the parser halts and announces the successful parsing.
4. If action $[s_m, a_i] = \text{error}$, it calls an error recovery routine.

Q38. Explain in detail about the construction of SLR parsing tables.

Answer :

Construction of SLR Parsing Table

Given a grammar G , construct the parsing action function and goto function for SLR parsing table.

1. Construct the canonical collection of $LR(0)$ items from G , $C = \{I_0, I_1, \dots, I_n\}$.
2. Construct the state i from item I_i .
3. Define the parsing actions for state i as follows:
 - (i) If there is an item $[A \rightarrow \alpha \bullet a \beta]$ in I_i and goto $(I_i, a) = I_j$ then mark the entry, action $[i, a] = \text{shift } j$ i.e., shift to state j where a is a terminal.
 - (ii) If there is an item $[A \rightarrow \alpha \bullet]$ in I_i and $A \neq S'$ then mark the entry action $[i, a] = \text{reduce } A \rightarrow \alpha$ for all terminals a in FOLLOW(A).
 - (iii) If there is an item $[S' \rightarrow \bullet S]$ in I_i then mark the entry action $[i, \$] = \text{accept}$.
4. Define the goto transitions for state i for all non-terminals A as,

if goto $(I_i, A) = I_j$ then mark the entry goto $[i, A] = j$
5. Mark the remaining blank entries as error entries.

The state constructed from the set of items containing $[S' \rightarrow \bullet S]$ is an initial state of the parser.

The parsing table constructed using above algorithm is called the $SLR(1)$ table for G . An LR parser that uses this table is called $SLR(1)$ parser for G . The grammar for which $SLR(1)$ parsing table can be constructed is said to be $SLR(1)$ grammar. A grammar is not a $SLR(1)$ if the parsing action function contains multiply-defined entries.

Example

Consider the augmented expression grammar.

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

For remaining answer refer Unit-IV, Page No. 122, Q36, Topic: Construction of Canonical Collection of $LR(0)$ Items (Only Example).

First consider the set $LR(0)$ of items I_0 .

$$I_0: E' \rightarrow \bullet E$$

$$E \rightarrow \bullet E + T$$

$$E \rightarrow \bullet T$$

$$T \rightarrow \bullet T * F$$

$$T \rightarrow \bullet F$$

$$F \rightarrow \bullet (E)$$

$$F \rightarrow \bullet id$$

In these sets of items we can apply the rule 3(i) to only two items $F \rightarrow \bullet (E)$ and $F \rightarrow \bullet id$ which results in entries into action table as action $[0, (] = \text{shift } 4$ and action $[0, id] = \text{shift } 5$.

Now consider the set of items I_1 .

$$I_1: E' \rightarrow E \bullet$$

$$E \rightarrow E \bullet + T$$

Here, the item $E' \rightarrow E \bullet$ results in action $[1, \$] = \text{accept}$ by applying rule 3(iii). Another item $E \rightarrow E \bullet + T$ results in action $[1, +] = \text{shift } 6$ by applying rule 3(i).

Now consider the set of items I_2 .

$$I_2: E \rightarrow T \bullet$$

$$T \rightarrow T \bullet * F$$

For first item, according to rule 3(ii) find FOLLOW(E) which is given as,

$$\text{FOLLOW}(E) = \{+,), \$\}$$

Therefore the actions for first item are action $[2,)] = \text{action } [2, \$] = \text{action } [2, +] = \text{reduce } E \rightarrow T$. The second item results in action $[2, *] = \text{shift } 7$. Continuing this process for all items in C we obtain the SLR parsing table as shown in the table below.

In the table, reduce action entries contains the numbers of productions as follows:

$$E \rightarrow E + T \quad \dots (1)$$

$$E \rightarrow T \quad \dots (2)$$

$$T \rightarrow T * F \quad \dots (3)$$

$$T \rightarrow F \quad \dots (4)$$

$$F \rightarrow (E) \quad \dots (5)$$

$$F \rightarrow id \quad \dots (6)$$

State Number	Action						Goto		
	Id	+	*	(	)	\$	E	T	F
0	shift 5			shift 4			1	2	3
1		shift 6				accept			
2		reduce 2	shift 7		reduce 2	reduce 2			
3		reduce 4	reduce 4		reduce 4	reduce 4			
4	shift 5			shift 4			8	2	3
5		reduce 6	reduce 6		reduce 6	reduce 6			
6	shift 5			shift 4				9	3
7	shift 5			shift 4					10
8		shift 6			shift 11				
9		reduce 1	shift 7		reduce 1	reduce 1			
10		reduce 3	reduce 3		reduce 3	reduce 3			
11		reduce 5	reduce 5		reduce 5	reduce 5			

Table: SLR Parsing Table for Expression Grammar

Q39. What are viable prefixes? Explain in detail.

Answer :

Viable Prefixes

Viable prefixes can be defined as the prefix (of the right-sentential form) that are acquired by the stack of a shift-reduce parser for the grammar G . These prefixes are named so because they always allow addition of terminal symbols as their suffixes to form a right-sentential form.

The viable prefixes forms the basis for SLR parsing where the $LR(0)$ automaton accepts viable prefixes.

For instance, consider the following,

$$X \rightarrow \beta_1 \beta_2$$

$$S \xrightarrow{rm} \alpha \times W$$

Then,

$$S \Rightarrow \alpha \beta_1 \beta_2 W$$

If $\alpha \beta_1$ is a viable prefix then $X \rightarrow \beta_1 \beta_2$ forms a valid item for $\alpha \beta_1$. However, an item can be valid for number of viable prefixes.

Now, with a valid item for a viable prefix it is easy to decide whether to perform shift or reduce action which as follows,

- If $\beta_2 \neq \epsilon$, then it represents that handle is not shifted onto the stack and thus shift action has to be performed.
- If $\beta_2 = \epsilon$, then it represents $X \rightarrow \beta_1$ as the handle and thus reduce operation has to be performed.

Thus, for a single viable prefix, there is a possibility of two valid items directing to perform different actions thereby leading to conflicts. However, these conflicts are solvable by looking at the next input-symbol or by other method.

The number of valid items for a viable prefix can be evaluated easily. The only step is to find the number of items reached from the first state along the path named γ in the $LR(0)$ automaton of the grammar.

4.3.7 More Powerful LR Parsers

Q40. Explain in detail about more powerful LR parsers.

Answer :

The more powerful LR parsers are as follows,

1. Canonical $LR(1)$ Parser

A $LR(1)$ includes an extra information, a terminal symbol as a second component in a state to identify the terminals symbols that can actually follow a handle for which there is a possible reduction to A i.e., there is a production $A \rightarrow \alpha$.

The general form of an $LR(1)$ item is $[A \rightarrow \alpha \cdot \beta, a]$ where $A \rightarrow \alpha \cdot \beta$ is a $LR(0)$ item and a is a set of terminal symbols. The '1' in $LR(1)$ stands for length of the second component i.e., a and it is called the look ahead symbol.

If an item is of the form $[A \rightarrow \alpha \cdot \beta, a]$ where $\beta \neq \epsilon$ then look ahead has no effect.

If an item is of the form $[A \rightarrow \alpha \cdot, a]$ then reduction to $A \rightarrow \alpha$ is applicable if and only if 'a' is the next input symbol.

An algorithm for the construction of the sets of $LR(1)$ items 'C' consists of the following steps.

1. Initially C contains the closure $(S' \rightarrow \cdot S, \$)$.
2. For each set of items I present in C and each grammar symbol X do step 3.
3. If goto (I, X) is not empty and it is not present in C then add goto (I, X) to C.
4. Repeat step 3 until there are no more sets of items that can be added to C.

In the algorithm, the closure and goto functions are defined as follows.

Closure Function

1. If set of items I contains an item $[A \rightarrow \alpha \cdot B \beta, a]$ and there is a production $B \rightarrow \gamma$ in G' and further the terminal b in $\text{FIRST}(\beta a)$ then, add the item $[B \rightarrow \cdot \gamma, b]$ to I, if it does not exist already.
2. Repeat the above step until there are no new items that can be added to closure (I).

Goto Function

The goto function is defined as follows.

If there are set of items $[A \rightarrow \alpha X \cdot \beta, a]$ such that there is an item $[A \rightarrow \alpha \cdot X \beta, a]$ in I then

$$\text{goto}([A \rightarrow \alpha \cdot X \beta, a]X) = \text{Closure}([A \rightarrow \alpha X \cdot \beta, a])$$

Example

Let us build the canonical collection of sets of $LR(1)$ items for the following augmented grammar.

$$S' \rightarrow S$$

$$S \rightarrow L \mid R \mid R$$

$$L \rightarrow \cdot R \mid id$$

$$R \rightarrow L$$

Now construct set of $LR(1)$ items I.

$$I_0 = \text{closure} \{[S' \rightarrow \cdot S, \$]\}$$

$$I_0: S' \rightarrow \cdot S, \$$$

$$S \rightarrow \cdot L = R, \$$$

$$S \rightarrow \cdot R, \$$$

$$L \rightarrow \cdot \cdot R, \$$$

$$L \rightarrow id, \$$$

$$R \rightarrow \cdot L, \$$$

$$L \rightarrow \cdot \cdot R, =$$

$$L \rightarrow \cdot id, =$$

I_0 can be rewritten as,

$$I_0: S' \rightarrow \cdot S, \$$$

$$S \rightarrow \cdot L = R, \$$$

$$S \rightarrow \cdot R, \$$$

$$L \rightarrow \cdot \cdot R, \$ | =$$

$$L \rightarrow \cdot id, \$ | =$$

$$R \rightarrow \cdot L, \$$$

Now find goto (I_0, X) where $X = S, L, R, \cdot$ and id .

$$I_1: \text{goto}(I_0, S) I_2: \text{goto}(I_0, L)$$

$$= \text{closure}(S' \rightarrow S \cdot, \$)$$

$$= \text{closure}(\{[S \rightarrow L \cdot = R, \$], [R \rightarrow L \cdot, \$]\})$$

$$: S' \rightarrow S \cdot, \$: S \rightarrow L \cdot R = R, \$$$

$$R \rightarrow L \cdot, \$$$

No more items can be added to closure (I) as the dot is at the right end.

$$I_3: \text{goto}(I_0, R)$$

$$= \text{closure}(S \rightarrow R \cdot, \$)$$

$$: S \rightarrow R \cdot, \$$$

$$I_4: \text{goto}(I_0, \cdot)$$

$$= \text{closure}(L \rightarrow \cdot \cdot R, \$ | =)$$

$$L \rightarrow \cdot \cdot R, = | \$$$

$$R \rightarrow \cdot L, = | \$$$

$$L \rightarrow \cdot \cdot R, = | \$$$

$$L \rightarrow \cdot id, = | \$$$

$$I_5: \text{goto}(I_0, id)$$

$$= \text{closure}(L \rightarrow id \cdot, \$ | =)$$

$$: L \rightarrow id \cdot, \$ | =$$

$$I_6: S \rightarrow L = \cdot R, \$$$

$$R \rightarrow \cdot L, \$$$

$$L \rightarrow \cdot \cdot R, \$$$

$$L \rightarrow \cdot id, \$$$

$$I_7: L \rightarrow \cdot R \cdot, \$ | =$$

$$I_8: R \rightarrow L \cdot, = | \$$$

$$I_9: S \rightarrow L = R \cdot, \$ | =$$

$$I_{10}: R \rightarrow L \cdot, \$$$

$$I_{11}: L \rightarrow id \cdot, \$$$

$$I_{12}: L \rightarrow \cdot \cdot R, \$$$

$$R \rightarrow \cdot L, \$$$

$$L \rightarrow \cdot \cdot R, \$$$

$$L \rightarrow \cdot id, \$$$

$I_0: L \rightarrow * R \cdot, \$$

Transition diagram for LR(1) items is shown below.

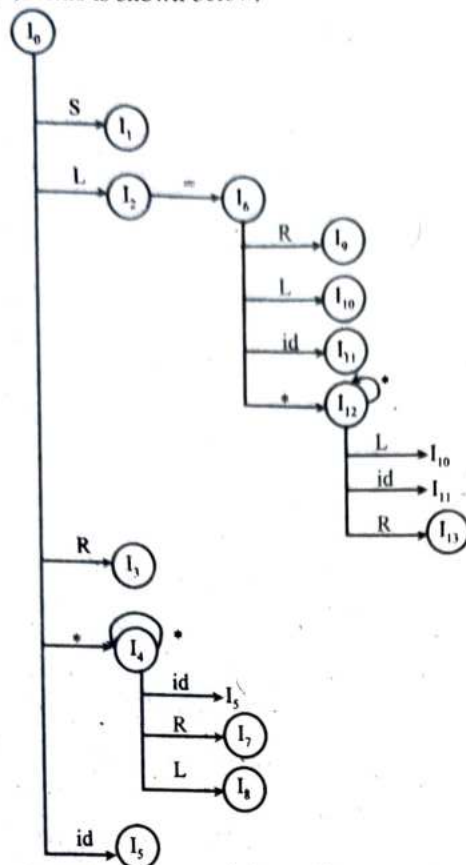


Figure: Transition Diagram for LR(1) Items

Q41. Discuss in detail about canonical LR(1) parsing tables.

Answer :

Canonical LR(1) Parsing Table

An algorithm for the construction of canonical parsing table is as follows,

1. Construct the canonical collection of LR(1) items for G' , $C = \{I_0, I_1, \dots, I_n\}$.
2. Construct the state i from item I_i .
3. Define the parsing actions for state i as follows.
 - (i) If there is an item $[A \rightarrow \alpha \cdot a \beta, b]$ in I_i and $\text{goto}(I_i, a) = I_j$ then mark the entry action $[i, a] = \text{shift } j$ i.e., shift to state j , where a is a terminal.
 - (ii) If there is an item $[A \rightarrow \alpha \cdot]$ in I_i and $A \neq S'$ then mark the entry action $[i, a] = \text{reduce } A \rightarrow \alpha$.
 - (iii) If there is an item $[S' \rightarrow S \cdot]$ in I_i then mark the entry action $[i, \$] = \text{accept}$.
4. Define the goto transitions for state i for all nonterminals A as, if $\text{goto}(I_i, A) = I_j$ then mark the entry $\text{goto}[i, A] = j$.
5. Mark the remaining blank entries as error entries.

The state constructed from the set of items containing $[S' \rightarrow \cdot S, \$]$ is an initial state of the parser.

The parsing table constructed using above algorithm is called the canonical LR(1) or CLR(1) parsing table for G . An LR parser that uses this table is called a CLR(1) parser for G . A grammar is said to be LR(1) grammar if the action function does not contain multiple-defined entries.

Example

Consider the following grammar,

$$S \rightarrow L = R$$

$$S \rightarrow R$$

$$L \rightarrow * R \mid id$$

$R \rightarrow L$

An augmented grammar for the above grammar is,

 $S' \rightarrow S$
 $S \rightarrow L = R \mid R$
 $L \rightarrow * R \mid id$
 $R \rightarrow L$

For remaining answer refer Unit-IV, Page No. 126, Q40, Topic: Example.

In the table, reduce action entries containing the numbers of productions as follows.

 $S \rightarrow L = R$
 $S \rightarrow R$
 $L \rightarrow * R$
 $L \rightarrow id$
 $R \rightarrow L$

State Number	Action				Goto		
	*	=	id	\$	S	L	R
0	shift 4		shift 5		1	2	3
1				accept			
2		shift 6		reduce 5			
3				reduce 2			
4	shift 4		shift 5			8	7
5		reduce 4		reduce 4			
6	shift 12		shift 11			10	9
7		reduce 3		reduce 3			
8		reduce 5		reduce 5			
9				reduce 1			
10				reduce 5			
11				reduce 4			
12	shift 12		shift 11			10	13
13				reduce 3			

Table: Canonical LR(1) Parsing Table

Q42. Discuss in detail how LALR parsing tables are constructed efficiently.

Answer :

Efficient Construction of LALR Parsing

Efficient construction of LALR parsing table involves the following steps.

Initially, the items of LR(0) or LR(1) are represented by their kernel.

Next, the kernels of LALR(1) items are constructed. This is accomplished by considering the kernels of LR(0) items produced in step-1 by the method of propagation and spontaneous generation as follows,

(i) Spontaneous Generation

For LR(0)-items I having $A \rightarrow \alpha \cdot \beta$, a as its kernel item and $J = \text{GOTO}(I, X)$, the statement $\text{GOTO}(\text{CLOSURE}(\{A \rightarrow \alpha \cdot \beta, a\}), X)$

$[B \rightarrow \gamma \cdot \delta, b]$

irrespective of the lookahead a of kernel item $A \rightarrow \alpha \cdot \beta$. The lookahead b is called the spontaneously generated look ahead. In addition to this, the lookahead $\$$ for the item $S' \rightarrow S$ is also referred to as spontaneously generated lookahead.

(ii) Generation Through Propagation

The $\text{GOTO}(\text{CLOSURE}(\{A \rightarrow \alpha \cdot \beta, b\}), X)$ due to the presence of this lookahead in the lookaheads of kernel item $A \rightarrow \alpha \cdot \beta$. This lookahead b is called propagated lookahead.

This is because the lookahead b is propagated from $A \rightarrow \alpha \cdot \beta$ in I to $B \rightarrow \gamma \cdot \delta$ in J .

(iii) Finally, LALR(1) parsing table is constructed by applying closure function to each kernel and determining table entries through the method of CLR parsing table construction.

WARNING: Xerox/Photocopying of this book is a CRIMINAL act. Anyone found guilty is LIABLE to face LEGAL proceedings.

Example

To construct the kernels of LR(0), consider the following grammar.

 $S' \rightarrow S$
 $S \rightarrow L = R$
 $S \rightarrow R$
 $L \rightarrow *R$
 $L \rightarrow id$
 $R \rightarrow L$

The LR(0) items for the above grammar is,

 $I_0: S' \rightarrow S$
 $S \rightarrow L = R$
 $L \rightarrow *R$
 $L \rightarrow id$
 $R \rightarrow L$
 $I_1: S' \rightarrow S.$
 $I_2: S \rightarrow L = R$
 $R \rightarrow L.$
 $I_3: S \rightarrow R.$
 $I_4: L \rightarrow *R$
 $R \rightarrow L$
 $L \rightarrow *R$
 $L \rightarrow id$
 $I_5: L \rightarrow id$
 $I_6: S \rightarrow L = R$
 $R \rightarrow L$
 $L \rightarrow *R$
 $L \rightarrow id$
 $I_7: L \rightarrow *R.$
 $I_8: R \rightarrow L.$
 $I_9: S \rightarrow L = R.$

The kernels of these LR(0) items are,

 $I_0: S' \rightarrow S$
 $I_1: S' \rightarrow S.$
 $I_2: S \rightarrow L = R$
 $R \rightarrow L.$
 $I_3: S \rightarrow R.$
 $I_4: L \rightarrow *R$
 $I_5: L \rightarrow id$
 $I_6: S \rightarrow L = R$
 $I_7: L \rightarrow *R.$
 $I_8: R \rightarrow L.$
 $I_9: S \rightarrow L = R.$

Example

To determine the lookaheads, First, calculate $CLOSURE(\{[S' \rightarrow S, \$]\})$ which produces the following grammar

 $S' \rightarrow S, \$$
 $S \rightarrow L = R, \$$
 $S \rightarrow R, \$$
 $L \rightarrow *R, \$$
 $L \rightarrow id, \$$
 $R \rightarrow L, \$$

The lookahead # of kernel item $S' \rightarrow S$ is propagated to following items

 $S' \rightarrow S$ in I_1
 $S \rightarrow L = R$ in I_2
 $S \rightarrow R$ in I_3
 $L \rightarrow *R$ in I_4
 $L \rightarrow id$ in I_5
 $R \rightarrow L$ in I_8

The propagation and spontaneous generation of Lookaheads, are repeatedly applied to generate the kernel items of all the sets. This is shown in the following table.

Sets I_i	Items	Spontaneously Generated Lookaheads	Propagated Lookaheads Generated in Each Repetition		
			I	II	III
I_0	$S' \rightarrow S$	\$	\$	\$	\$
I_1	$S' \rightarrow S.$		\$	\$	\$
I_2	$S \rightarrow L = R$		\$	\$	\$
I_3	$S \rightarrow R.$		\$	\$	\$
I_4	$L \rightarrow *R$	=	= \$	= \$	= \$
I_5	$L \rightarrow id$	=	= \$	= \$	= \$
I_6	$S \rightarrow L = R$		\$	\$	\$
I_7	$L \rightarrow *R.$		=	= \$	= \$
I_8	$R \rightarrow L.$		=	= \$	= \$
I_9	$S \rightarrow L = R.$				\$

Table: Lookahead Generation

Q43. Explain the compaction of LR parsing tables with an example.

Answer :

Compaction of LR Parsing Tables

There are few techniques used to compress the ACTION and GOTO fields of an LR parsing table. The simplest technique used for compacting the action field is to recognize the identical multiple rows of the action table. This can be done by creating a pointer for each state into one dimensional array. This helps in saving the space and time parsing as the states with the same actions will point to the same location. To access the array information assign each terminal a number from zero to one less than the number of terminals. These integers are used as an offset from the value of the pointer for each state. The parsing action for i^{th} terminal will be found from the locations past the pointer value for that state.

Further the space can be increased at the expense of somewhat slower parser by creating a list for the actions of each state. The list contains (terminal-symbol, action) pairs. Place the most frequent action for a state at the end of the list, and use 'any' notation in place of α terminal. Error entries can be safely replaced by reduce actions.

Example

Consider the below table.

STATE id	ACTION						GOTO		
	+	*	(	)	\$		E	T	F
0	S5			S4			1	2	3
1		S6				accept			
2		r2	S7		r2	r2			
3		r4	r4		r4	r4			
4	S5			S4			8	2	3
5		r6	r6		r6	r6			
6	S5			S4				9	3
7	S5			S4					10
8		r3	r3		r3	r3			
9		r5	r5		r5	r5			

Action for states 0, 4, 6, 7 rows are identical represent all these by the list

SYMBOL	ACTION
id	S5
(	S4
any	error

State 1 has similar list:

+	S6
\$	accept
any	error

In state 2, replace the error entries by r2, so reduction by production 2 will occur on any input but *.

*	S7
any	r2

State 3 has only error and r4 entries. So replace before one with latter, so the state 3 contains (any, r4) pair in the list. States 5,8,9 can be treated in the same way.

Now encode the GOTO table by a list but it is better to make a list of pairs for each nonterminal A. For A, each pair on the list is of the form (current state, next state), indication is GOTO [current state, A] = next State.

As there are few states in any one column of GOTO table, this technique is useful.

WARNING: Xerox/Photocopying of this book is a CRIMINAL act. Anyone found guilty is LIABLE to face LEGAL proceedings.

Consider the same example,

F column has entry 10 for state 7 and all other entries are either 3 or error. Replace error by 3, the list is

CURRENT STATE	NEXT STATE
7	10
any	3

Similarly, for column T,

6	9
any	2

For E column, either 1 or 8 can be default. The list is

4	8
any	1

Q44. Compare and contrast between SLR, LALR and LR parsers.

Answer :

The bottom-up parsers differ in their size, capability and the class of grammars that they can handle. The comparison among SLR parser, canonical LR parser and LALR parser is as follows,

S.No.	SLR Parser	LR Parser	LALR Parser
1.	SLR parser is easier to implement.	LR parser is expensive in terms of time and space.	LALR parser is expensive in terms of time and space.
2.	The table obtained by it is smaller.	It has the largest size tables.	It has the same size as that of a SLR parser.
3.	It uses FOLLOW information for reductions.	It uses the lookahead symbols.	It uses the lookahead symbols.
4.	It can handle only few classes of grammar.	It can handle large sub classes of context free grammars.	It can handle a wider class of grammars than SLR grammar.
5.	It is the least powerful.	It is the most powerful.	It lies between SLR and LR parsers.
6.	Before announcing an error, it may make several reductions. But it never shift an erroneous input symbol onto the stack.	It can detect a syntactic error as soon as possible. While scanning the input from left to right.	It behaves similar to SLR parser.
7.	The table constructed by this parser is called the <i>SLR(1)</i> parsing table.	The table constructed by this parser is called the canonical <i>LR(1)</i> parsing table.	The table constructed by this parser is called the <i>LR(1)</i> parsing table.
8.	A grammar that has an <i>SLR(1)</i> parsing table is called <i>SLR(1)</i> grammar.	A grammar that has a canonical <i>LR(1)</i> parsing table is called <i>LR(1)</i> grammar.	A grammar that has a <i>LALR(1)</i> parsing table is called <i>LALR(1)</i> grammar.

VERY SHORT QUESTIONS WITH SOLUTIONS (VSQS)**Q1. Define compiler.****Answer :**

A compiler is a program that converts a source program into a target program.

Q2. What are the features of a Lexical analyser?**Answer :**

The features of lexical analyzer are as follows,

- (i) It removes white space and comments.
- (ii) It generates series of tokens.

Q3. Write down any two compiler construction tools.**Answer :**

The two compiler construction tools are,

1. Scanner generators
2. Code-generators.

Q4. Define context free grammar.**Answer :**

A context free grammar (or simply a grammar) describes the programming language constructs. It consists of terminals, non-terminals, a start symbol and productions. It is denoted by $G = (V, V_n, S, P)$.

Q5. Define parser.**Answer :**

A parser (or) syntax analyzer is the second phase of a compiler. It takes the tokens (such as keywords, identifier, ... etc.,) from the scanner or lexical analyzer and checks the syntax of the input stream (string of tokens) with the rules of a grammar(G) defined by the programming language.

Q6. What are the different types of bottom-up parsers?**Answer :**

The three different types of bottom-up parsers are as follows,

1. Shift-reduce parser
2. Precedence parser
3. LR parser.

Q7. Describe about canonical LR(0) collection.**Answer :**

A collection of set of LR(0) items is called as canonical LR(0) collection, this collection is used for construction of SLR parsers.

Q8. List the properties of LR parser.**Answer :**

The following are the various properties of LR parsers,

- (i) The programming languages for which CFG can be written are determined by constructing LR parsers.
- (ii) LR parsers generally operates via non backtracking shift reduce approach. Hence, it is considered to be efficient.

Q9. What do you mean by lexical analysis?**Answer :**

Lexical analysis is the process of reading the input string, identifying tokens, deleting white spaces and locating, repairing and reporting errors if there are any.

FREQUENTLY ASKED QUESTIONS & IMPORTANT QUESTIONS

Q1. With a neat diagram, explain the phases of a compiler in detail.

Important Question

Answer :

For answer refer Unit-IV, Page No. 96, Q2.

Q2. Explain compiler construction tools.

Important Question

Answer :

For answer refer Unit-IV, Page No. 99, Q4.

Q3. What are the functions of lexical analyzer?

Important Question

Answer :

For answer refer Unit-IV, Page No. 99, Q5.

Q4. Define lexeme, token and pattern. Identify the lexeme that make up the token in the following program segment. Indicate the corresponding token and pattern:

```
void swap(int i, int j)
```

```
{ int t;
```

```
  t = i;
```

```
  i = j;
```

```
  j = t;
```

```
}
```

Important Question

Answer :

For answer refer Unit-IV, Page No. 101, Q7.

Q5. What is meant by input buffering? Explain the use of sentinels in recognizing tokens.

Important Question

Answer :

For answer refer Unit-IV, Page No. 103, Q9.

Q6. What do you mean by token recognition? Explain in detail about transition diagram that is used to recognize the token relop (corresponding to relational operators in C++ language).

Important Question

Answer :

For answer refer Unit-IV, Page No. 104, Q11.

Q7. Write about lexical analyzer generator lex.

Important Question

Answer :

For answer refer Unit-IV, Page No. 106, Q13.

Q8. Define parser. Discuss about,

(a) Role of the parser

(b) Representative grammars.

Important Question

Answer :

For answer refer Unit-IV, Page No. 107, Q15.

Q9. Explain in detail about syntax error handling.

Important Question

Answer :

For answer refer Unit-IV, Page No. 108, Q16.

Q10. Discuss about the notational conventions of context free grammars.

Answer :

For answer refer Unit-IV, Page No. 109, Q18.

Important Question

Q11. Explain in detail about writing a grammar.

Answer :

For answer refer Unit-IV, Page No. 110, Q21.

Important Question

Q12. Discuss in detail Recursive-Descent parsing.

Answer :

For answer refer Unit-IV, Page No. 112, Q24.

Important Question

Q13. Explain the procedure for non-recursive predictive parsing table with an example.

Answer :

For answer refer Unit-IV, Page No. 116, Q28.

Important Question

Q14. Construct a predictive parsing table for the Grammar

$E \rightarrow E+T | T, T \rightarrow T^*F | F, F \rightarrow (E) | id.$

Answer :

For answer refer Unit-IV, Page No. 118, Q30.

Important Question

Q15. What is bottom-up parsing and reductions? Explain with example.

Answer :

For answer refer Unit-IV, Page No. 119, Q31.

Important Question

Q16. What is LR parsing? Also discuss its advantages and disadvantages.

Answer :

For answer refer Unit-IV, Page No. 121, Q35.

Important Question

Q17. Explain in detail about the construction of SLR parsing tables.

Answer :

For answer refer Unit-IV, Page No. 124, Q38.

Important Question

Q18. Explain in detail about more powerful LR parsers.

Answer :

For answer refer Unit-IV, Page No. 125, Q40.

Important Question

Q19. Discuss in detail how LALR parsing tables are constructed efficiently.

Answer :

For answer refer Unit-IV, Page No. 128, Q42.

Important Question

Q20. Compare and contrast between SLR, LALR and LR parsers.

Answer :

For answer refer Unit-IV, Page No. 131, Q44.

Important Question